

CSE 4621

Machine Learning

Topic: Neural Networks II

Source: [DeepLearning.AI](https://deeplearning.ai)

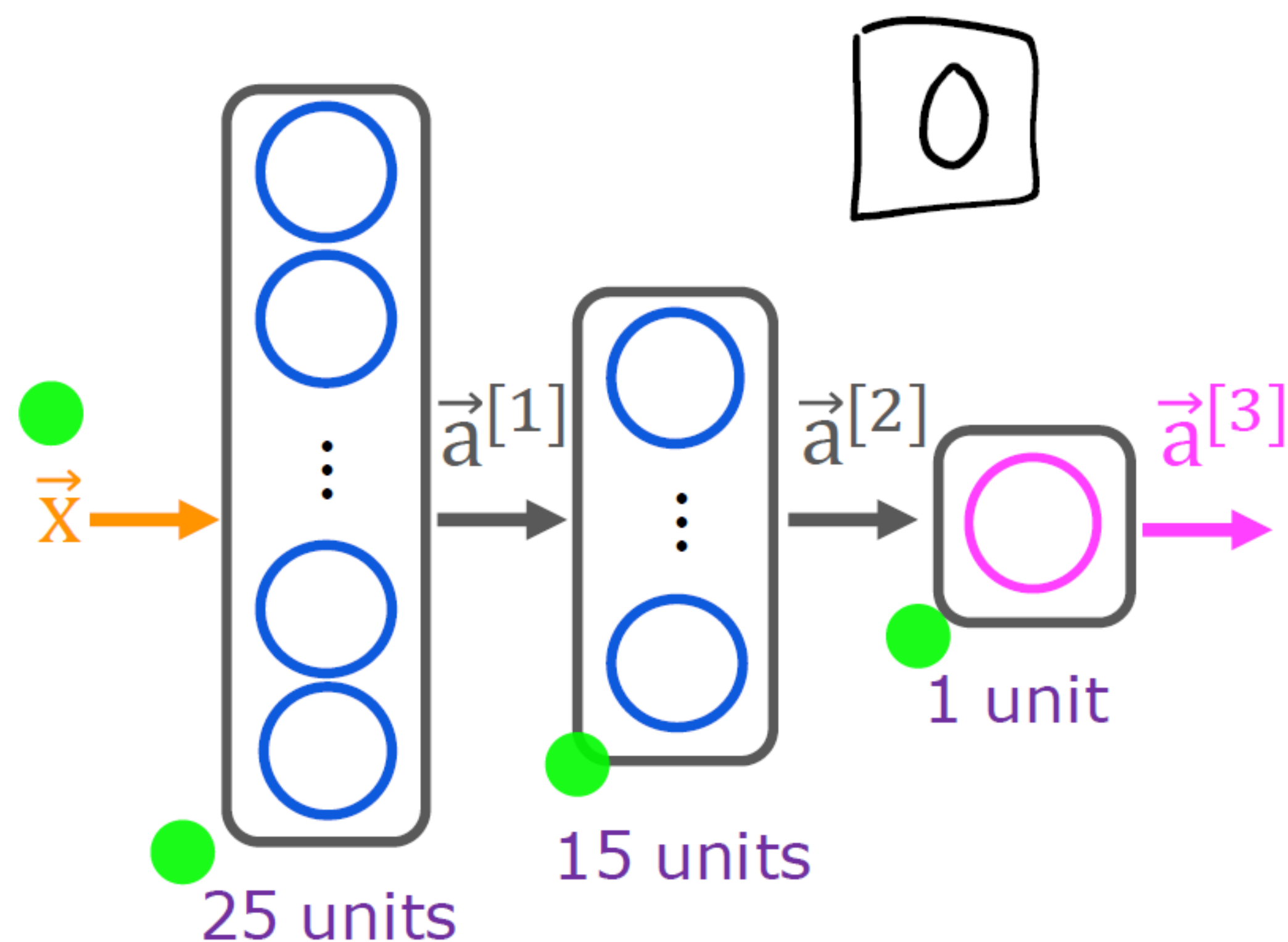
Presented by

Rafsanjany Kushol

Assistant Professor



Train a Neural Network in TensorFlow



Given set of (x, y) examples

How to build and train this in code?

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])

from tensorflow.keras.losses import BinaryCrossentropy

model.compile(loss=BinaryCrossentropy())

model.fit(X, Y, epochs=100)
```

epochs: number of steps
in gradient descent

Model Training Steps

①

specify how to compute output given input x and parameters w, b (define model)

$$f_{\vec{w}, b}(\vec{x}) = ?$$

②

specify loss and cost

$$L(f_{\vec{w}, b}(\vec{x}), y) \quad \text{1 example}$$

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$$

③

Train on data to minimize $J(\vec{w}, b)$

logistic regression

$$z = \text{np.dot}(w, x) + b$$

$$f_x = 1 / (1 + \text{np.exp}(-z))$$

logistic loss

$$\text{loss} = -y * \text{np.log}(f_x) - (1-y) * \text{np.log}(1-f_x)$$

$$w = w - \text{alpha} * \text{dj_dw}$$

$$b = b - \text{alpha} * \text{dj_db}$$

neural network

```
model = Sequential([
    Dense(...)
    Dense(...)
    Dense(...)
])
```

binary cross entropy

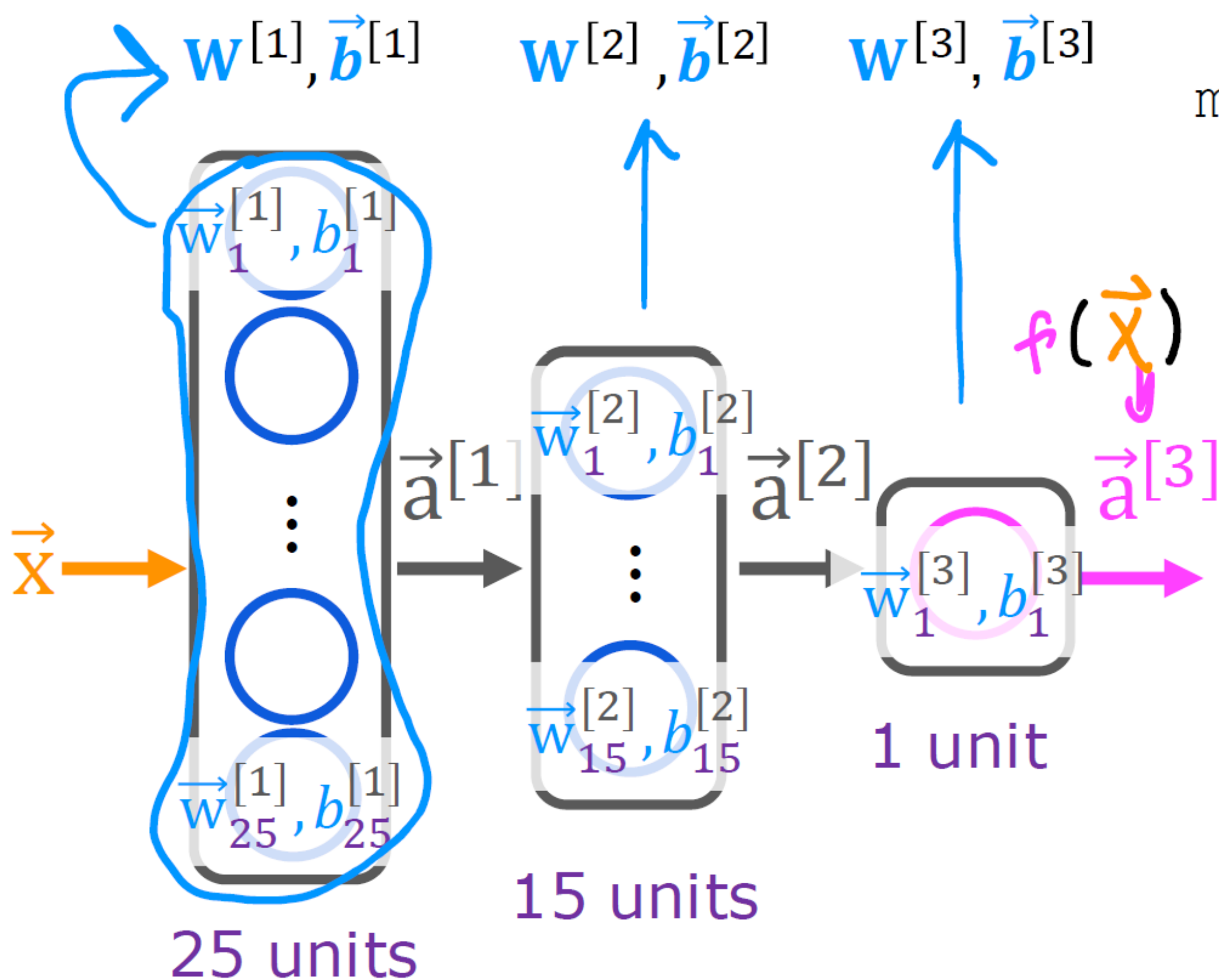
```
model.compile(
    loss=BinaryCrossentropy())
```

```
model.fit(X, y, epochs=100)
```


1. Create the model

define the model

$$f(\vec{x}) = ?$$



```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])
```

2. Loss and cost functions

Mnist digit classification problem binary classification

$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$

compare prediction vs. target

logistic loss

also known as binary cross entropy

$$J(\mathbf{W}, \mathbf{B}) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})$$

$\mathbf{W}^{[1]}, \mathbf{W}^{[2]}, \mathbf{W}^{[3]}$ $\vec{b}^{[1]}, \vec{b}^{[2]}, \vec{b}^{[3]}$ $f_{\mathbf{W}, \mathbf{B}}(\vec{x})$

```
model.compile(loss= BinaryCrossentropy())
```

```
from tensorflow.keras.losses import  
BinaryCrossentropy
```



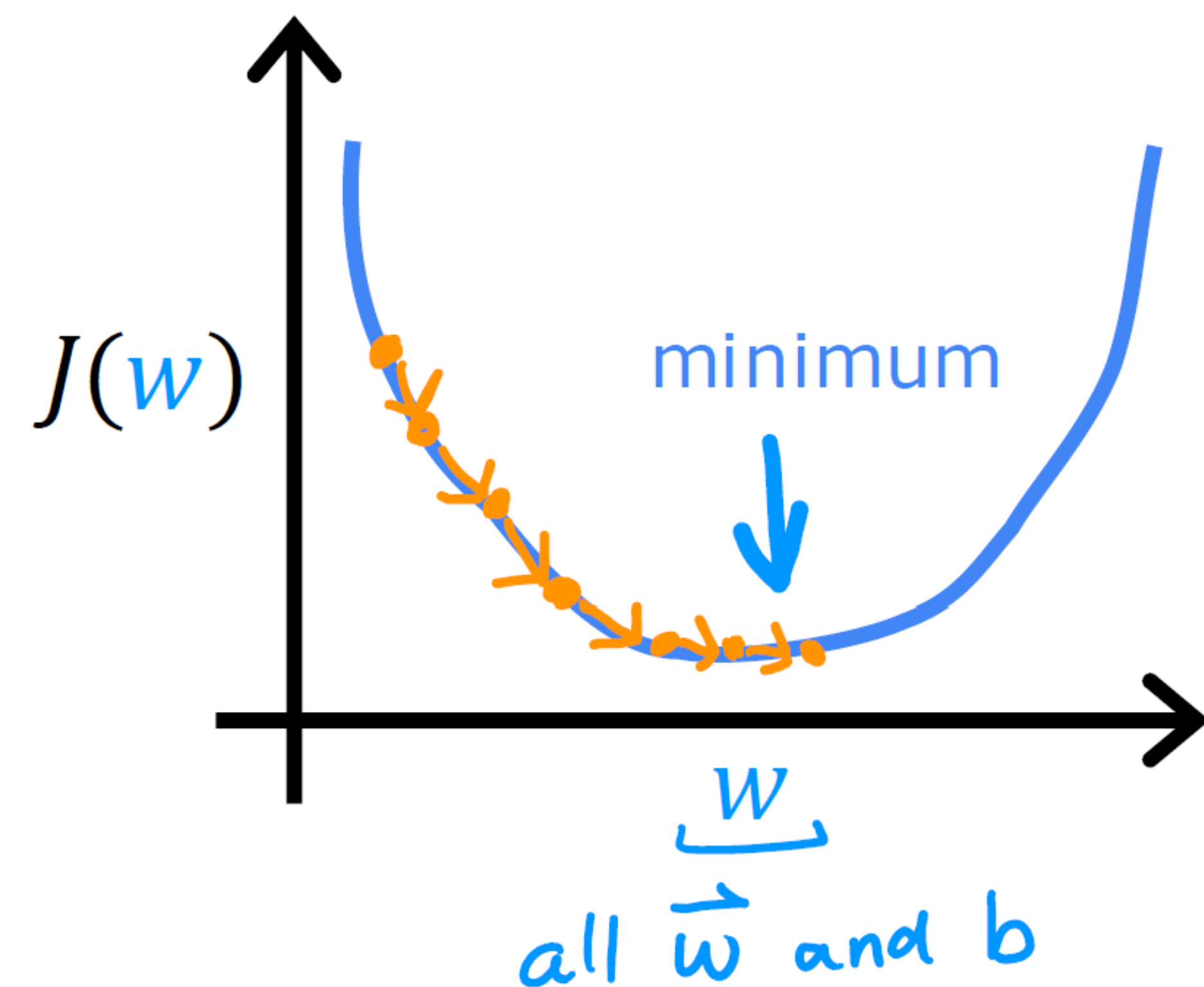
regression
(predicting numbers
and not categories)

mean squared error

```
model.compile(loss= MeanSquaredError())
```

```
from tensorflow.keras.losses import  
MeanSquaredError
```

3. Gradient descent



repeat {

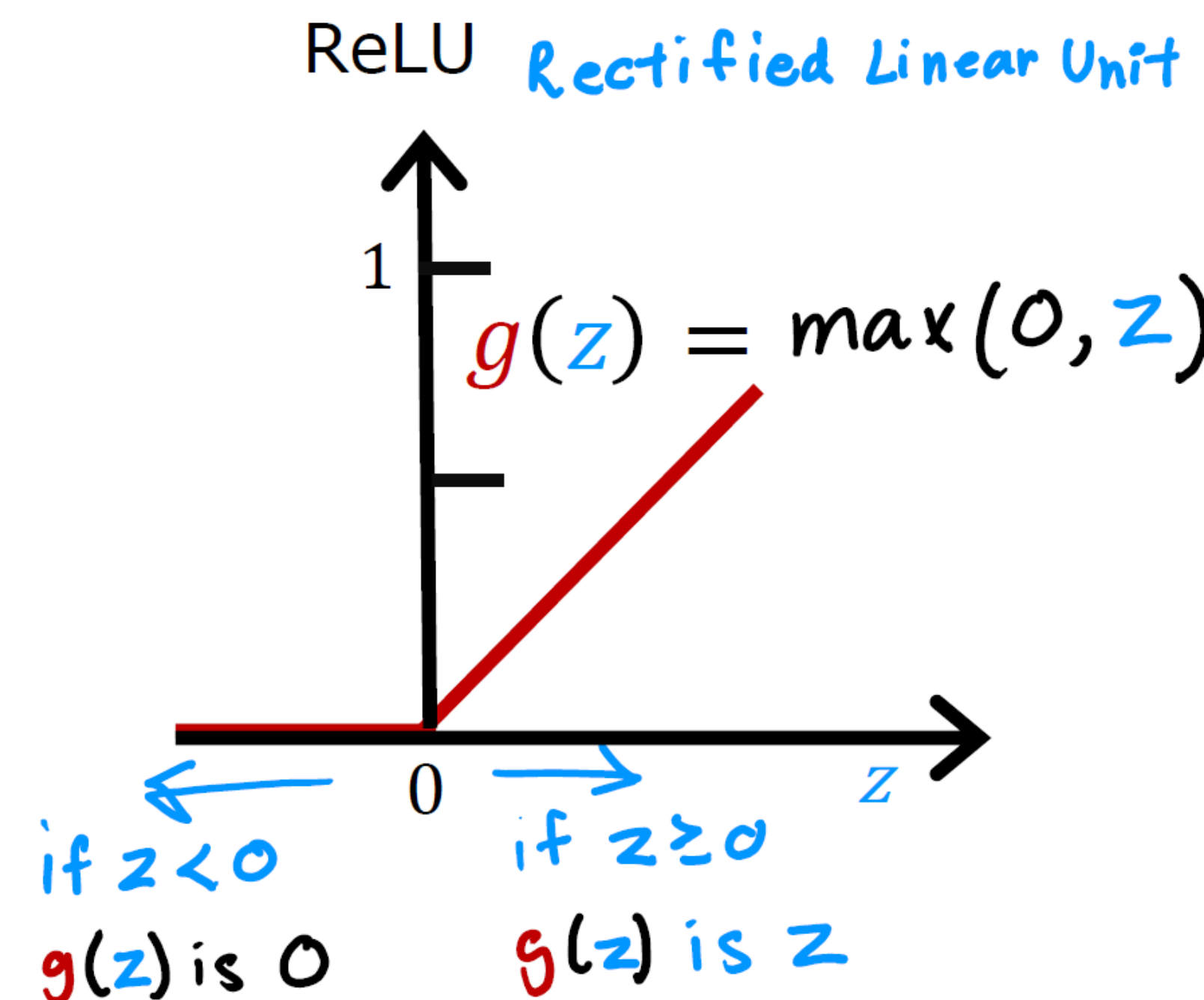
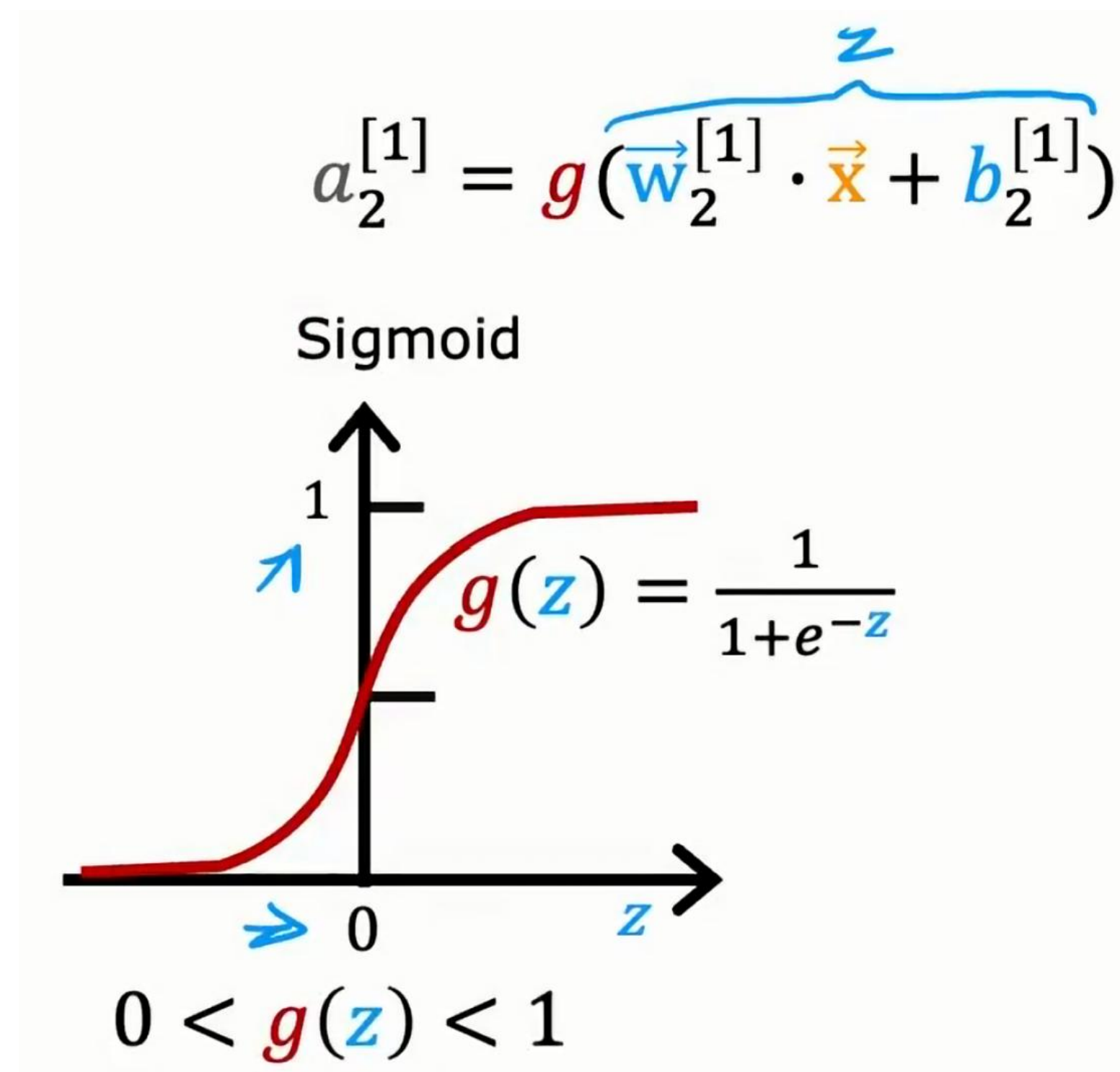
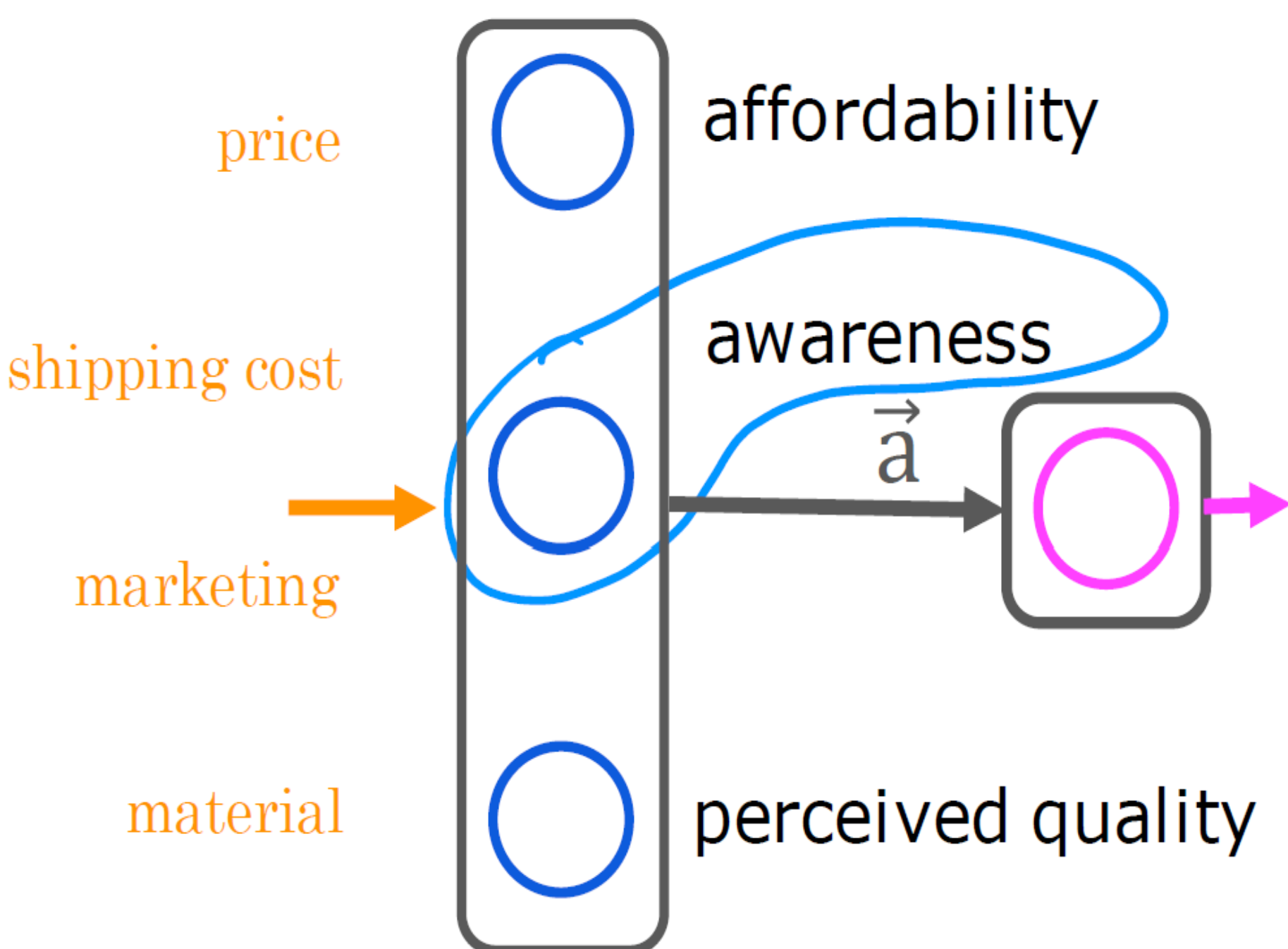
$$w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

} Compute derivatives
for gradient descent
using "backpropagation"

```
model.fit(X, y, epochs=100)
```

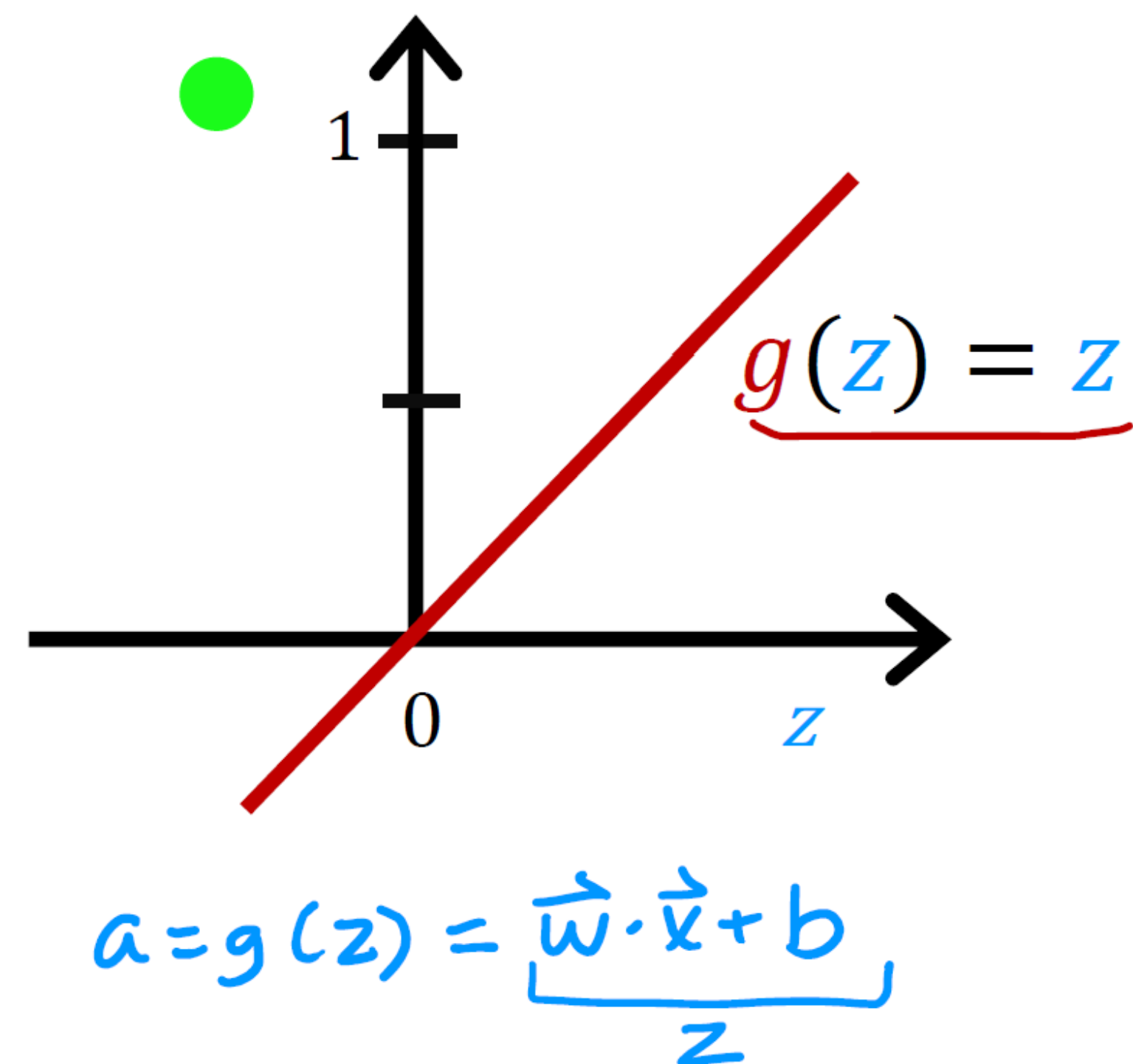

Activation Functions



Examples of Activation Functions

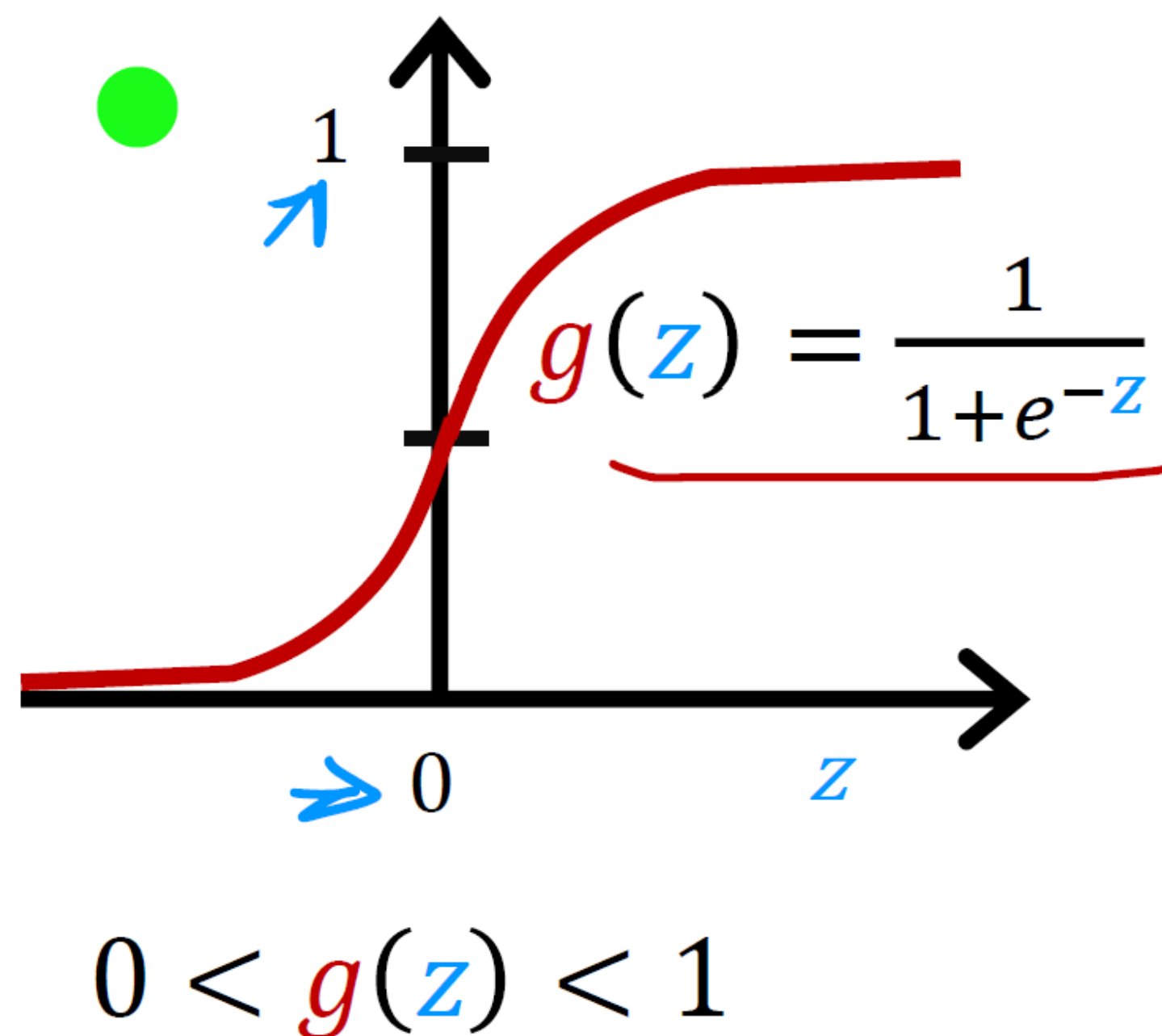
"No activation function"

Linear activation function



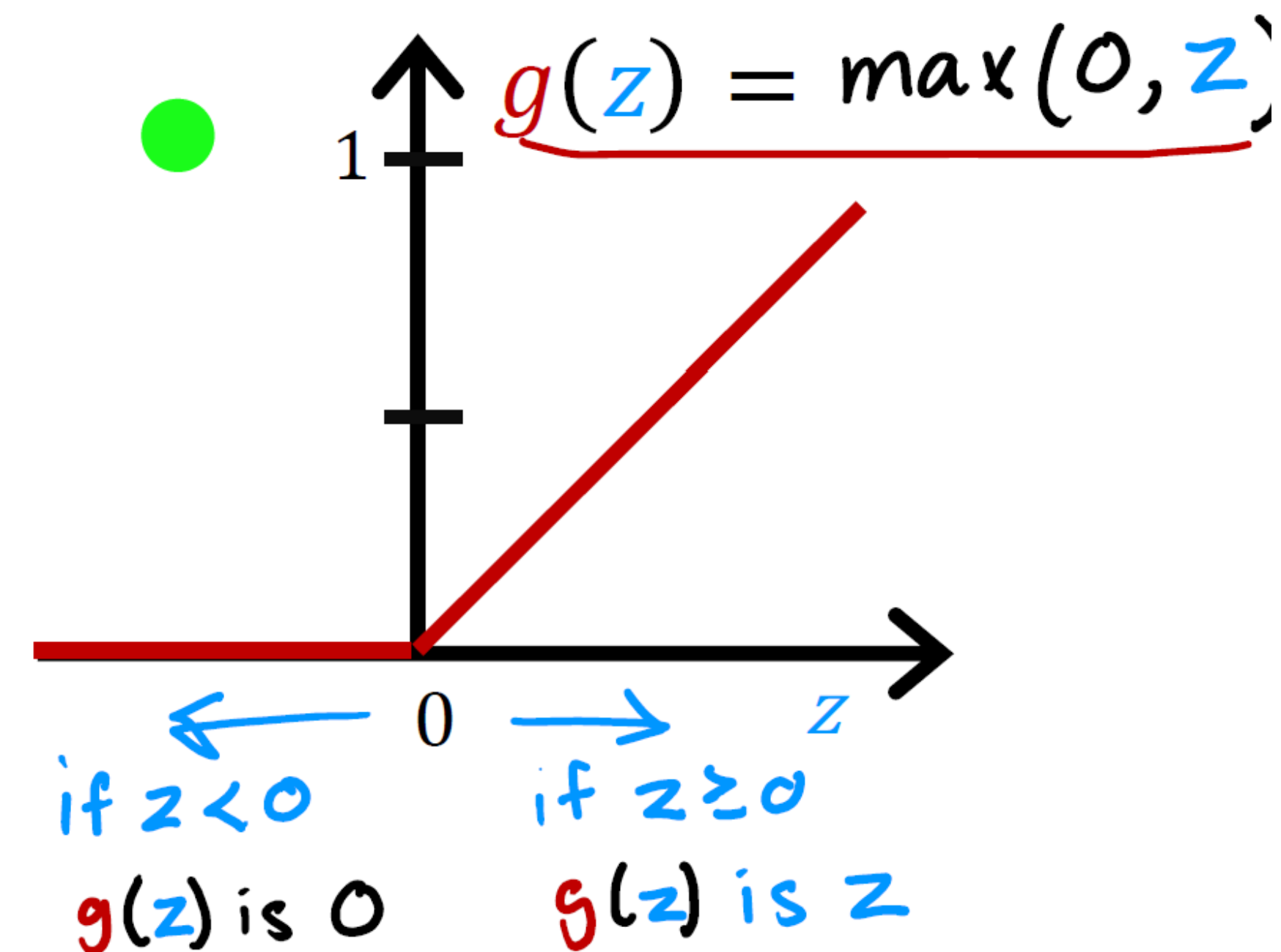
$$a_2^{[1]} = g(\overbrace{\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}}^z)$$

Sigmoid



Later: softmax activation

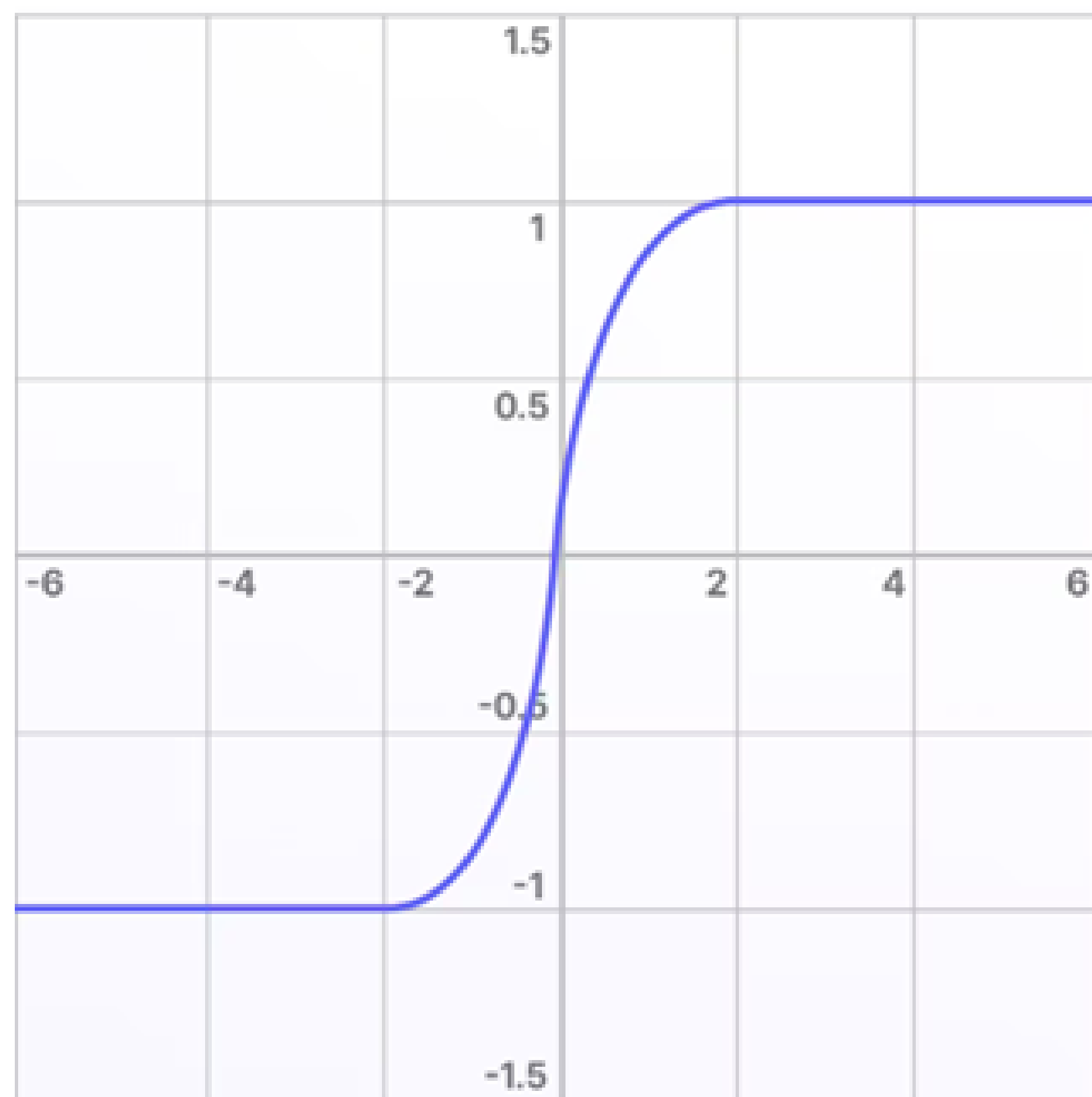
ReLU Rectified Linear Unit



Examples of Activation Functions

Tanh (Hyperbolic Tangent) Function

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Leaky ReLU

$$f(x) = \begin{cases} \alpha x, & x \leq 0 \\ x, & x > 0 \end{cases}$$



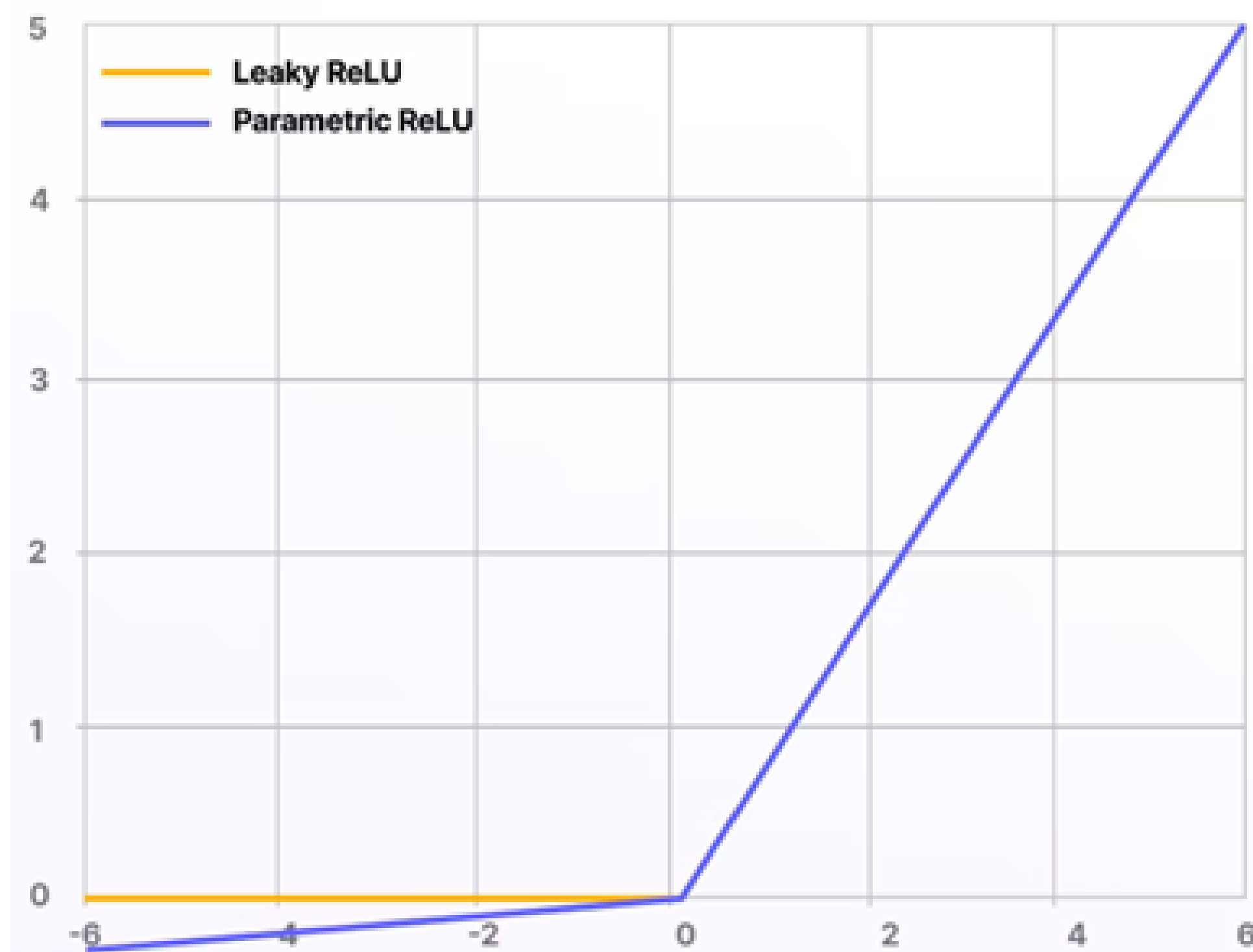
Examples of Activation Functions

Parametric ReLU (PReLU)

It is defined as:

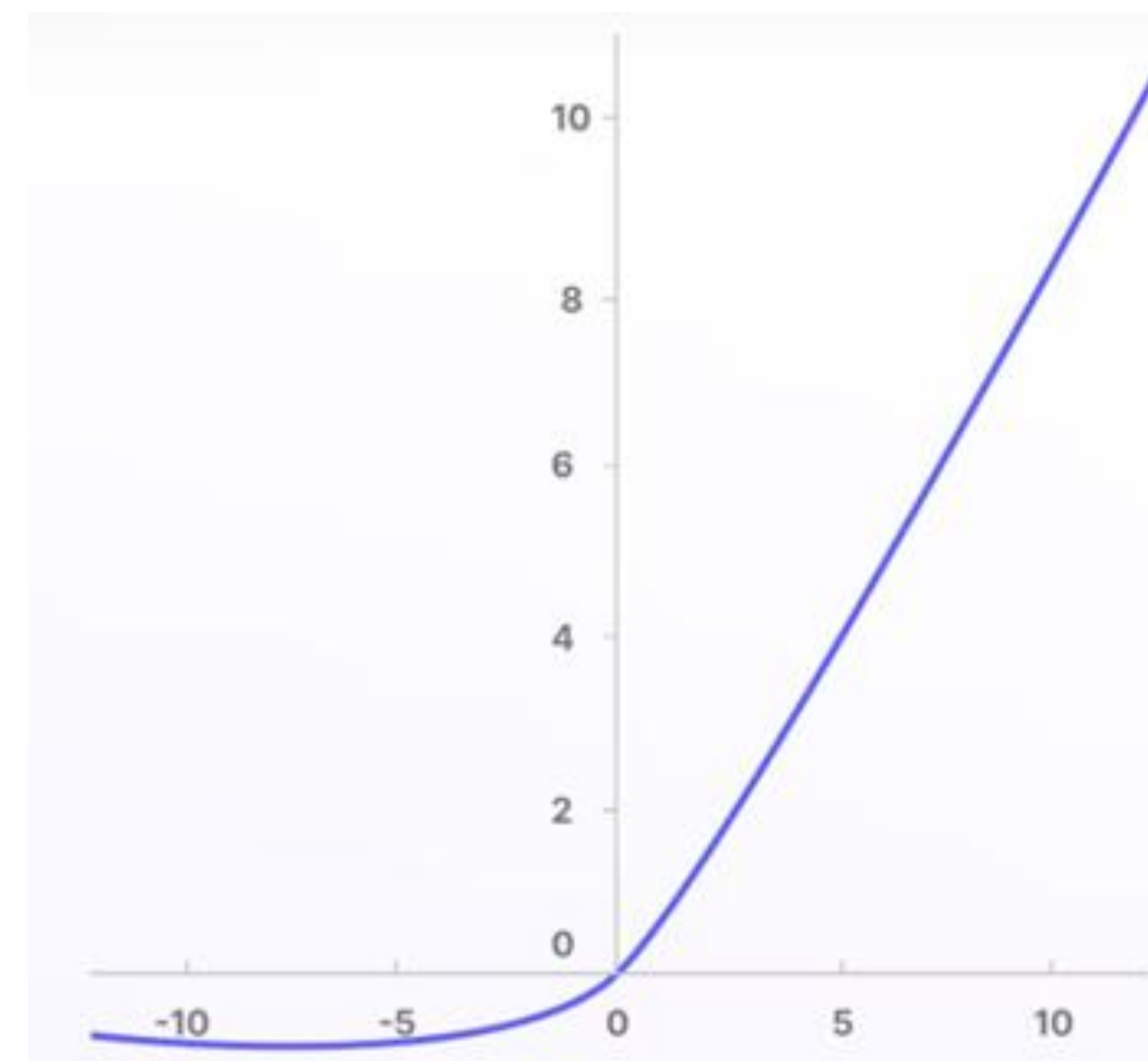
$$f(x) = \begin{cases} \alpha x, & x \leq 0 \\ x, & x > 0 \end{cases}$$

Leaky ReLU vs Parametric ReLU



Exponential Linear Unit (ELU)

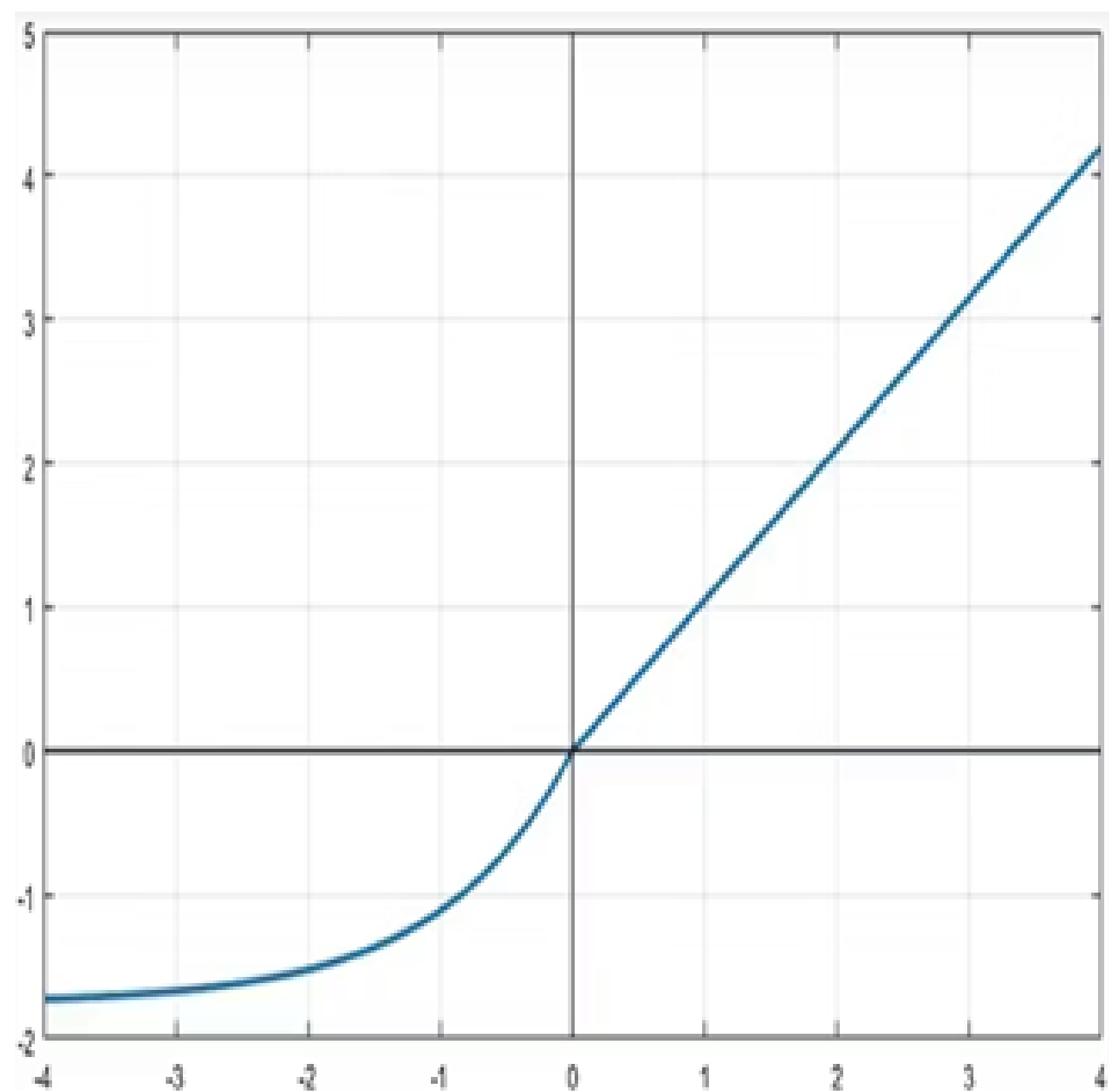
$$f(x) = \begin{cases} \alpha(e^x - 1), & x \leq 0 \\ x, & x > 0 \end{cases}$$



Examples of Activation Functions

Scaled Exponential Linear Unit (SELU)

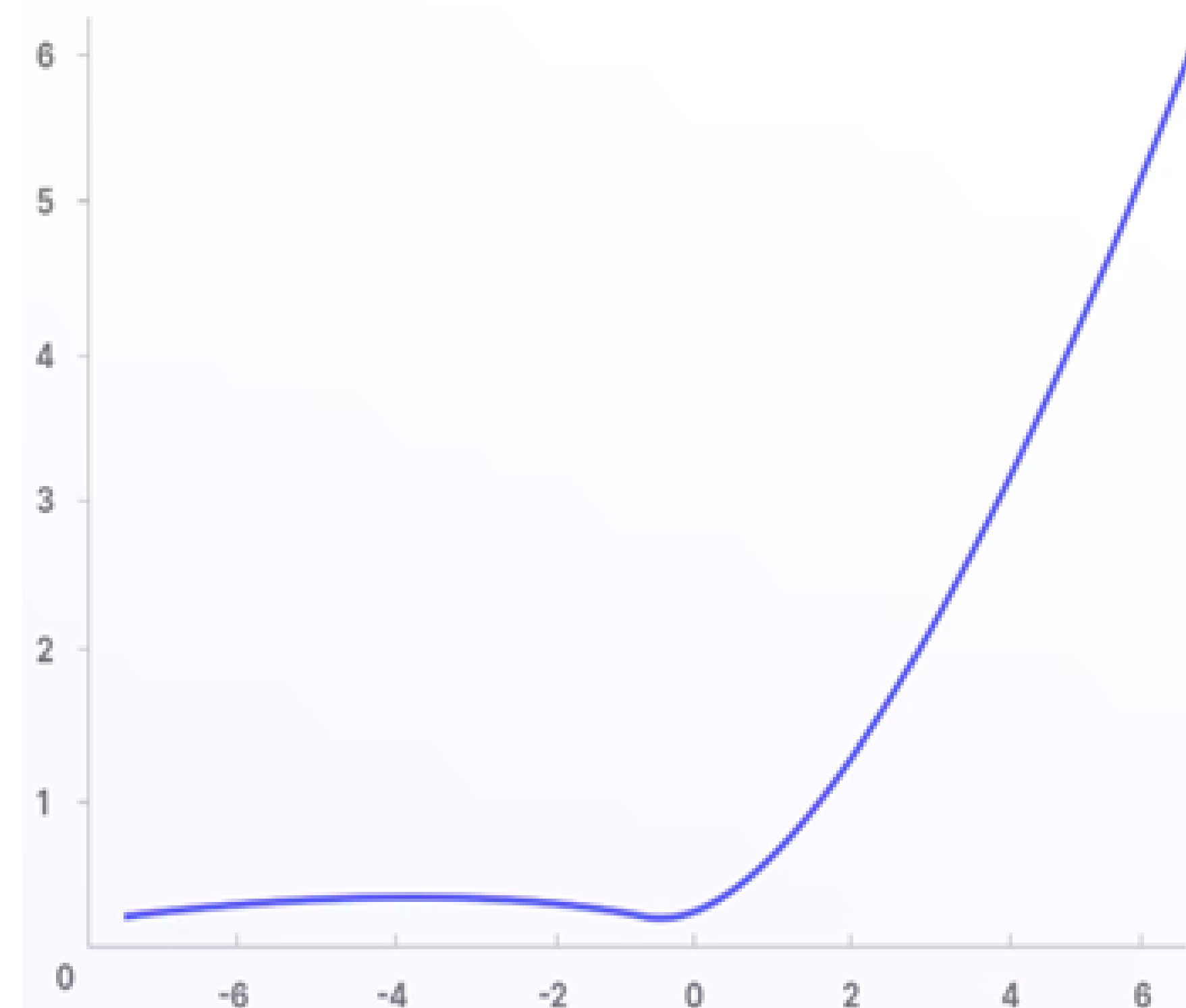
$$f(x) = \lambda \begin{cases} \alpha(e^x - 1), & x \leq 0 \\ x, & x > 0 \end{cases}$$



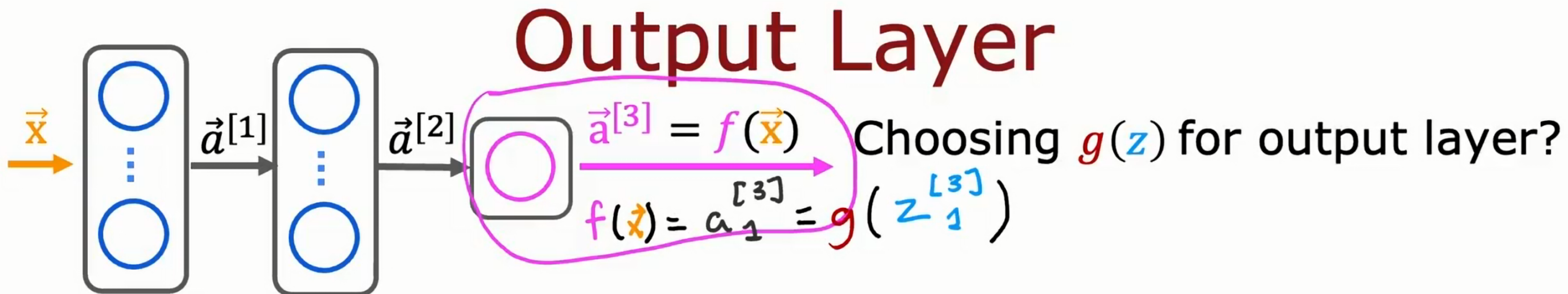
Gaussian Error Linear Unit (GELU)

$$f(x) = 0.5x \left(1 + \tanh \left(\sqrt{\frac{\pi}{2}} (x + 0.044715x^3) \right) \right)$$

Gaussian Error Linear Unit (GELU)

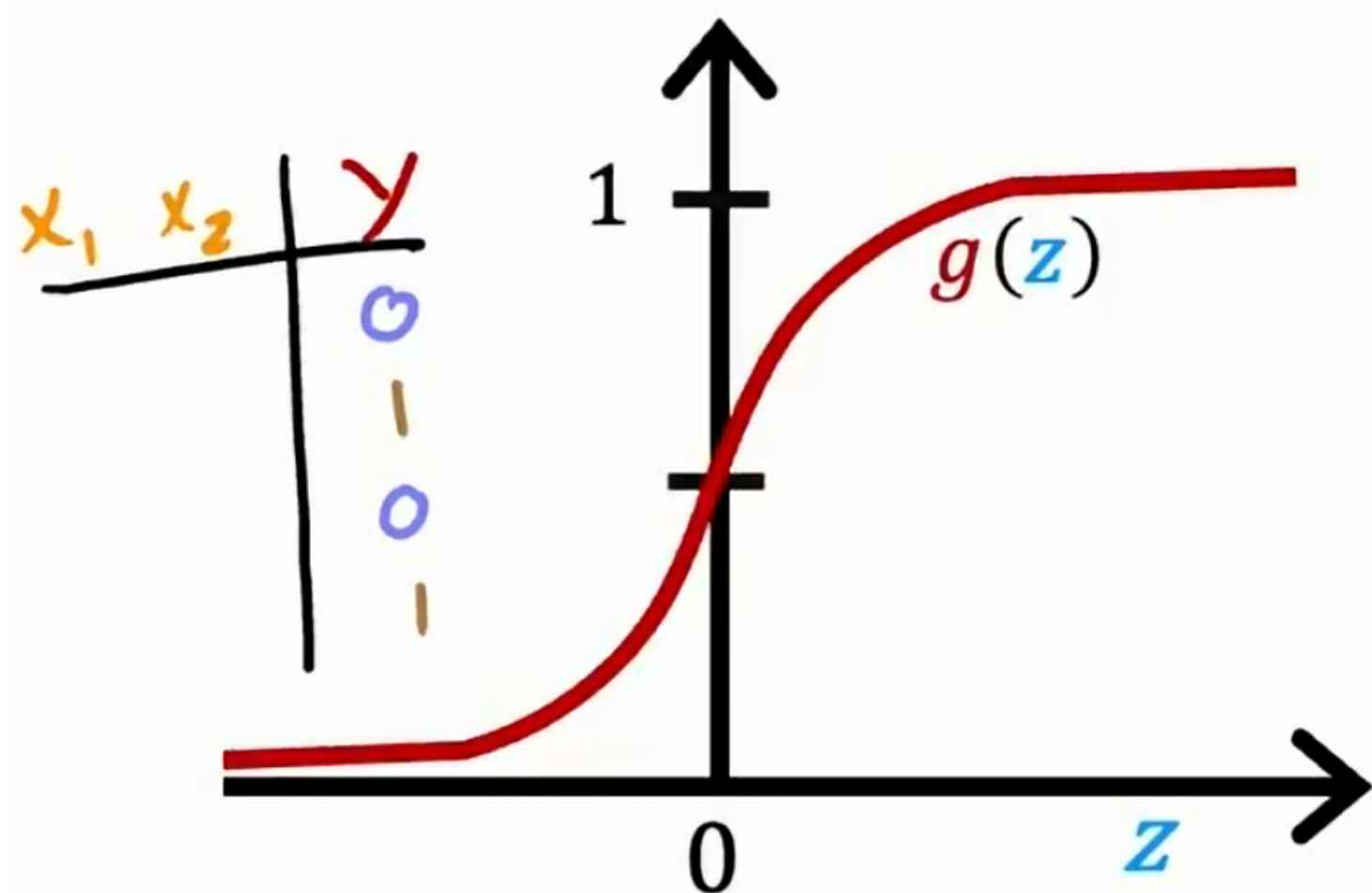


Choosing activation functions



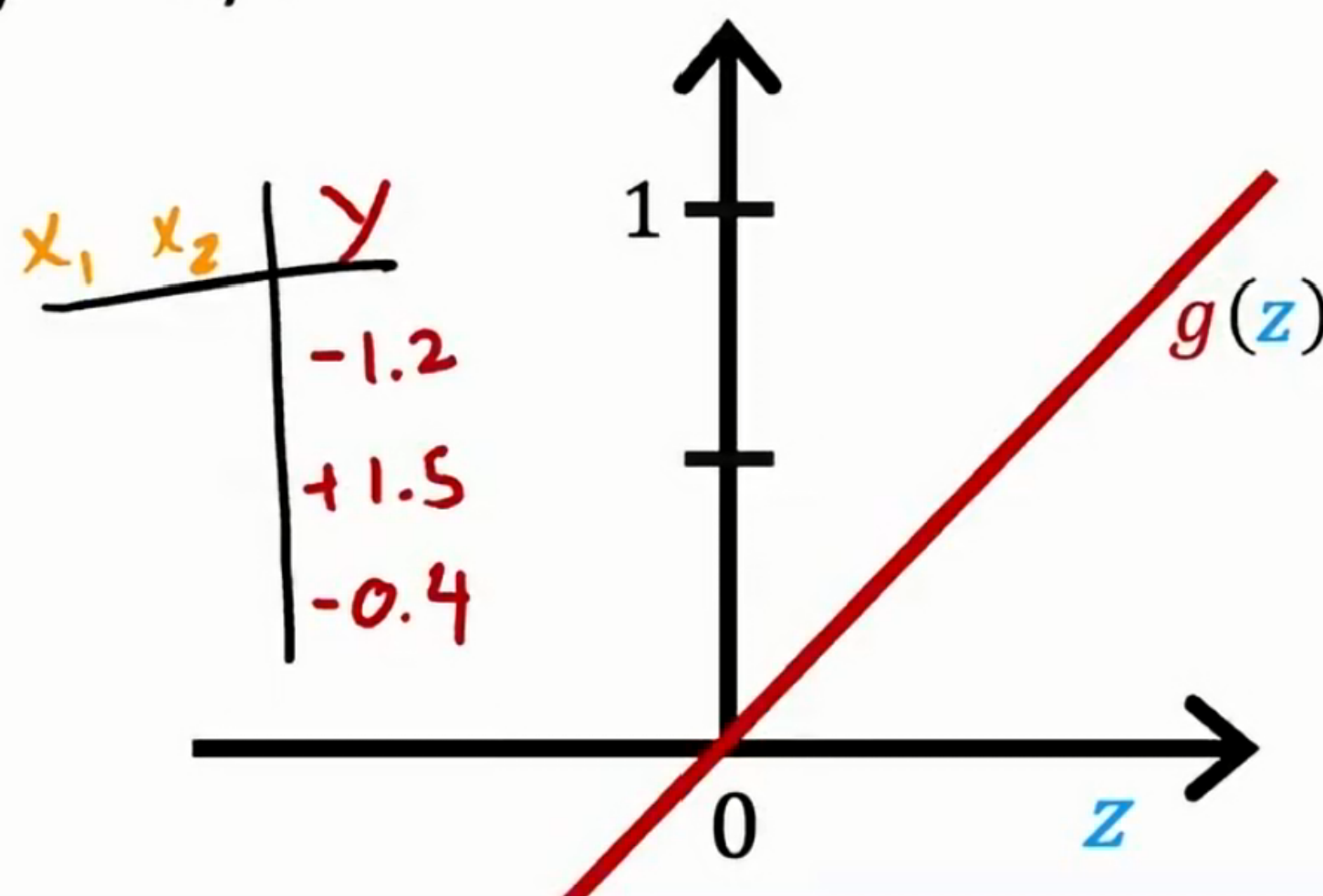
Binary classification

Sigmoid
 $y=0/1$



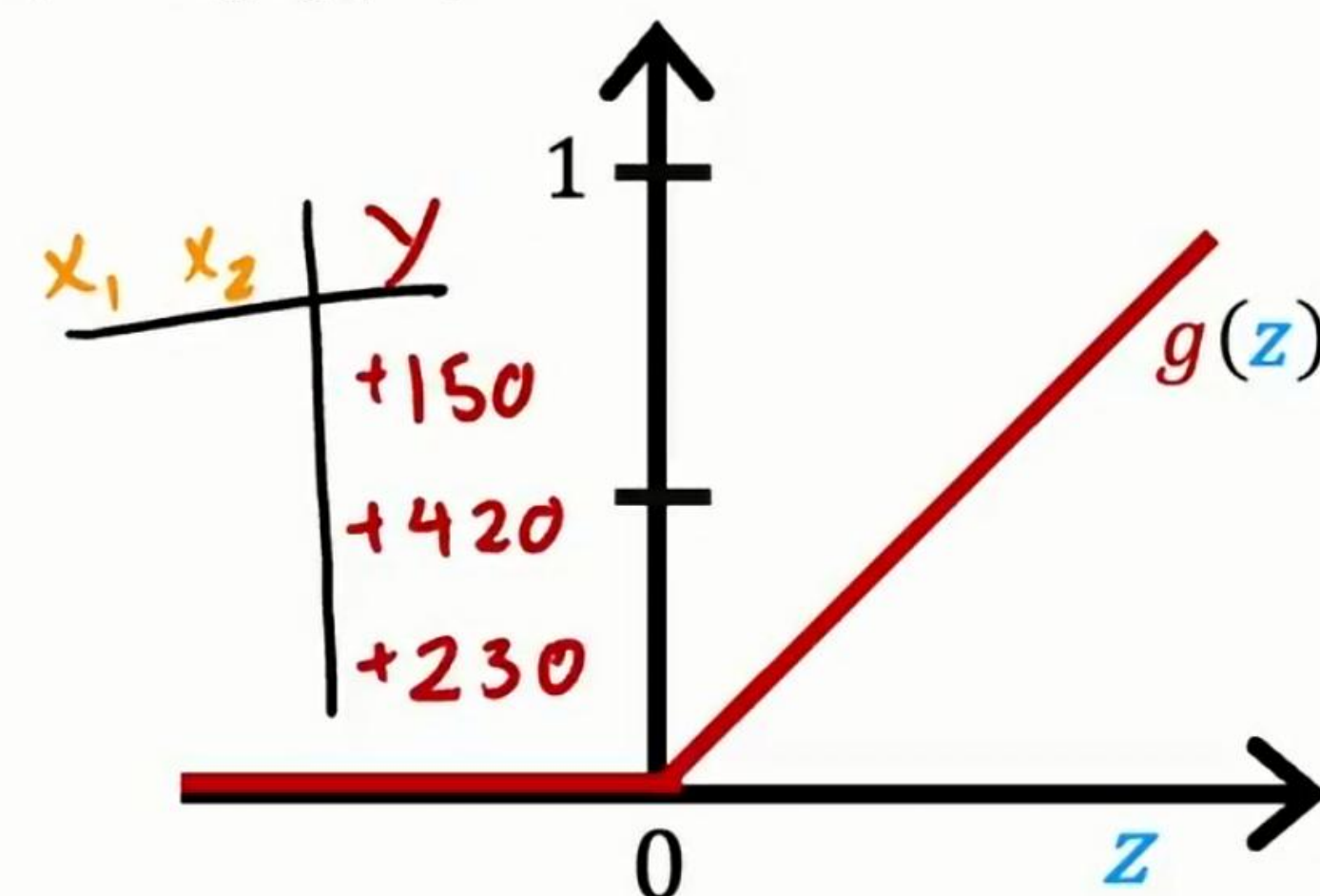
Regression

Linear activation function
 $y = +/-$



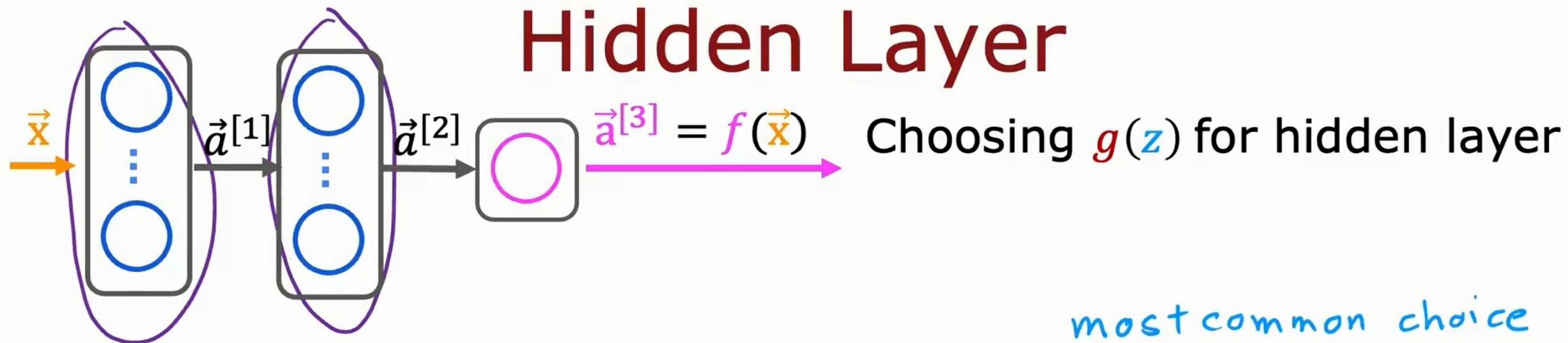
Regression

ReLU
 $Y = 0$ or $+$

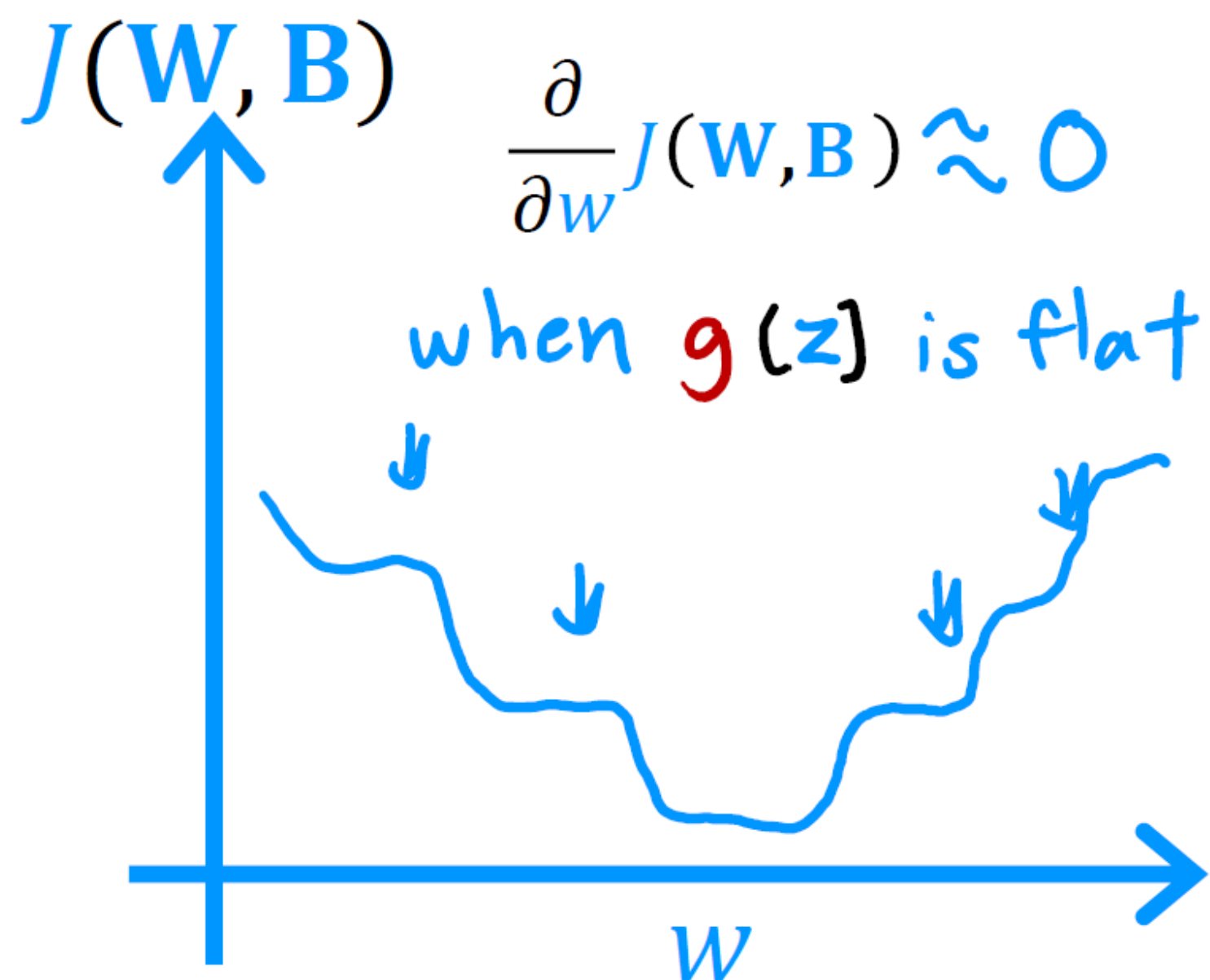
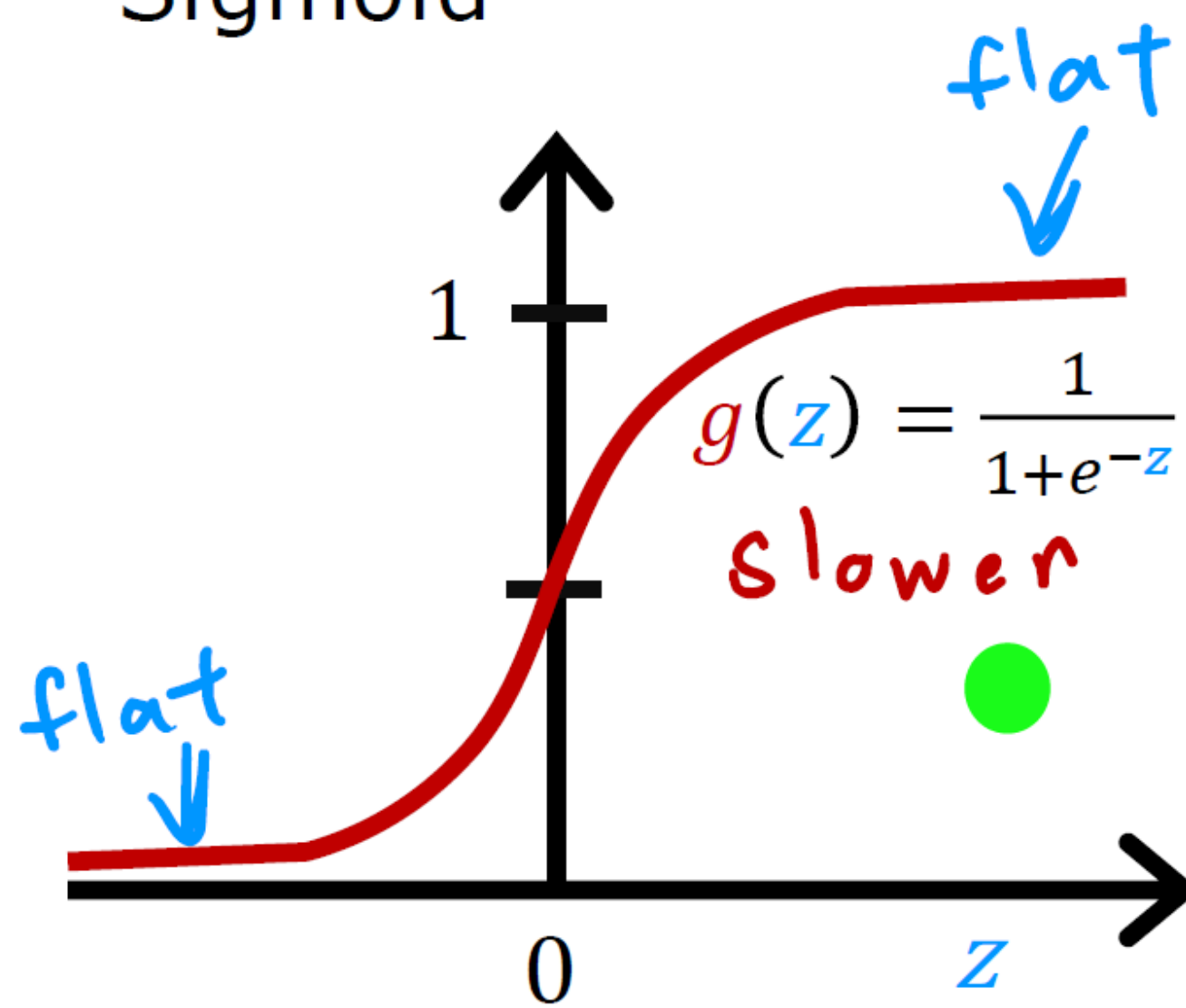


Choosing activation functions

Hidden Layer



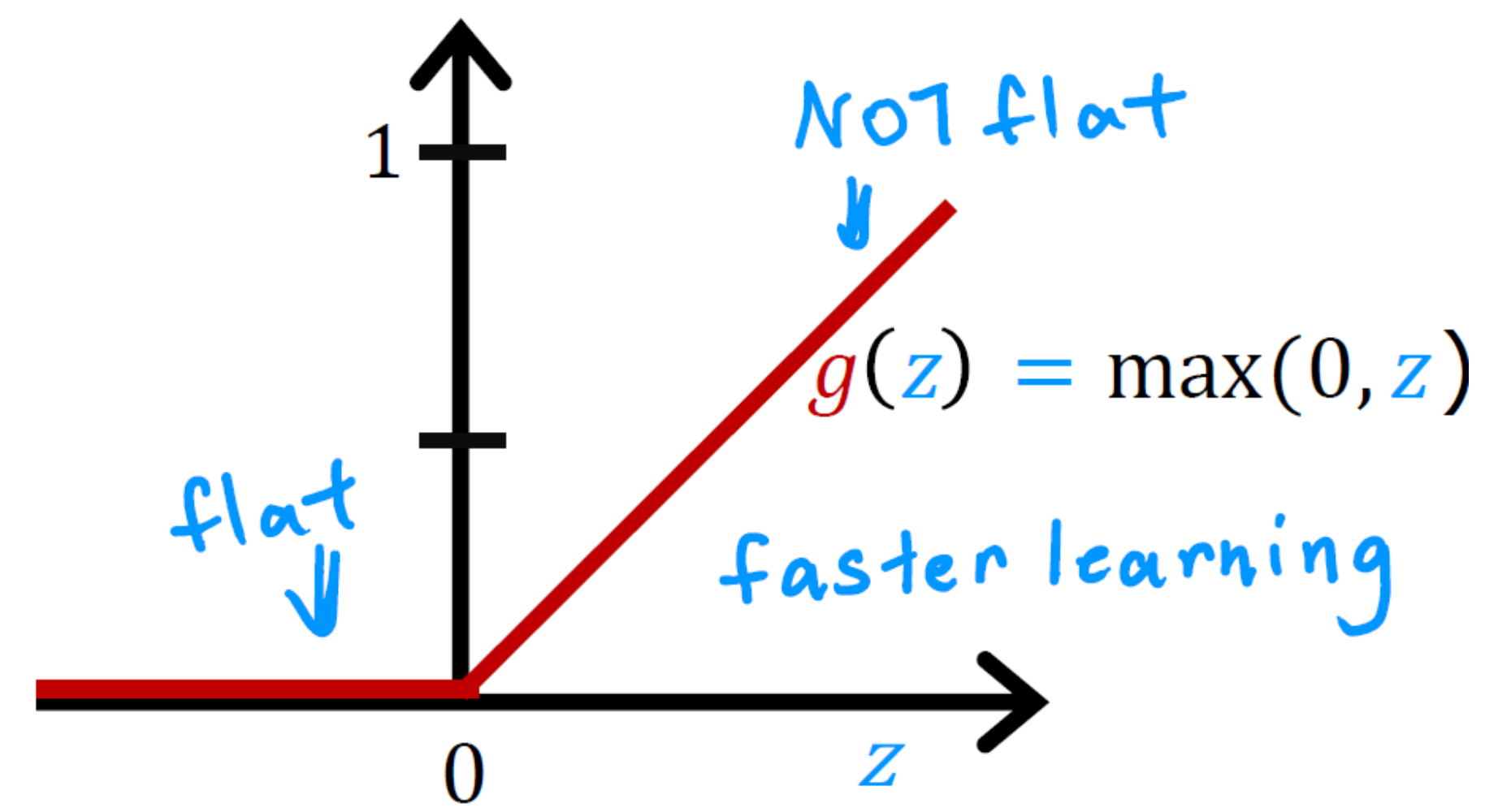
Sigmoid



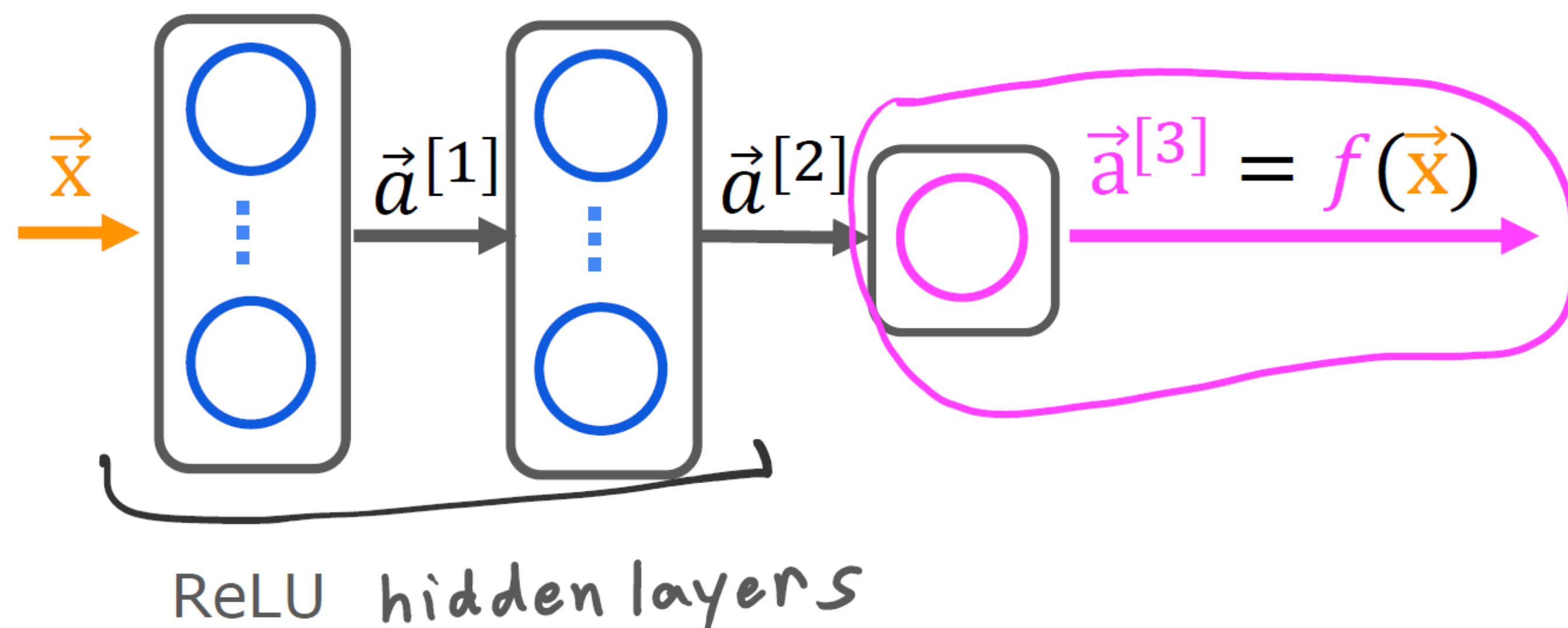
most common choice

ReLU

faster



Choosing activation summary



binary classification

activation='sigmoid'

regression y negative / positive
activation='linear'

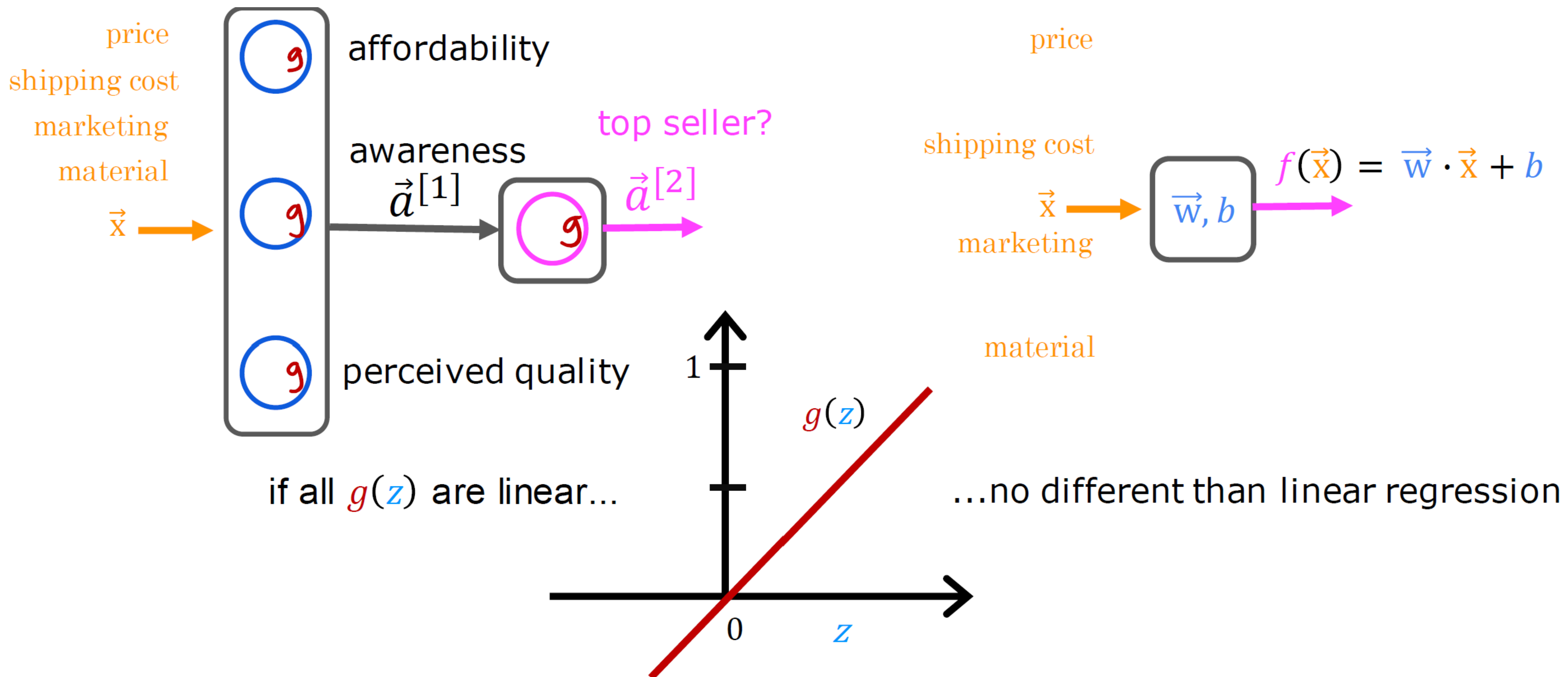
regression $y \geq 0$

activation='relu'

```
from tf.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),    layer1
    Dense(units=15, activation='relu'),    layer2
    Dense(units=1, activation='sigmoid')   layer3
])
```

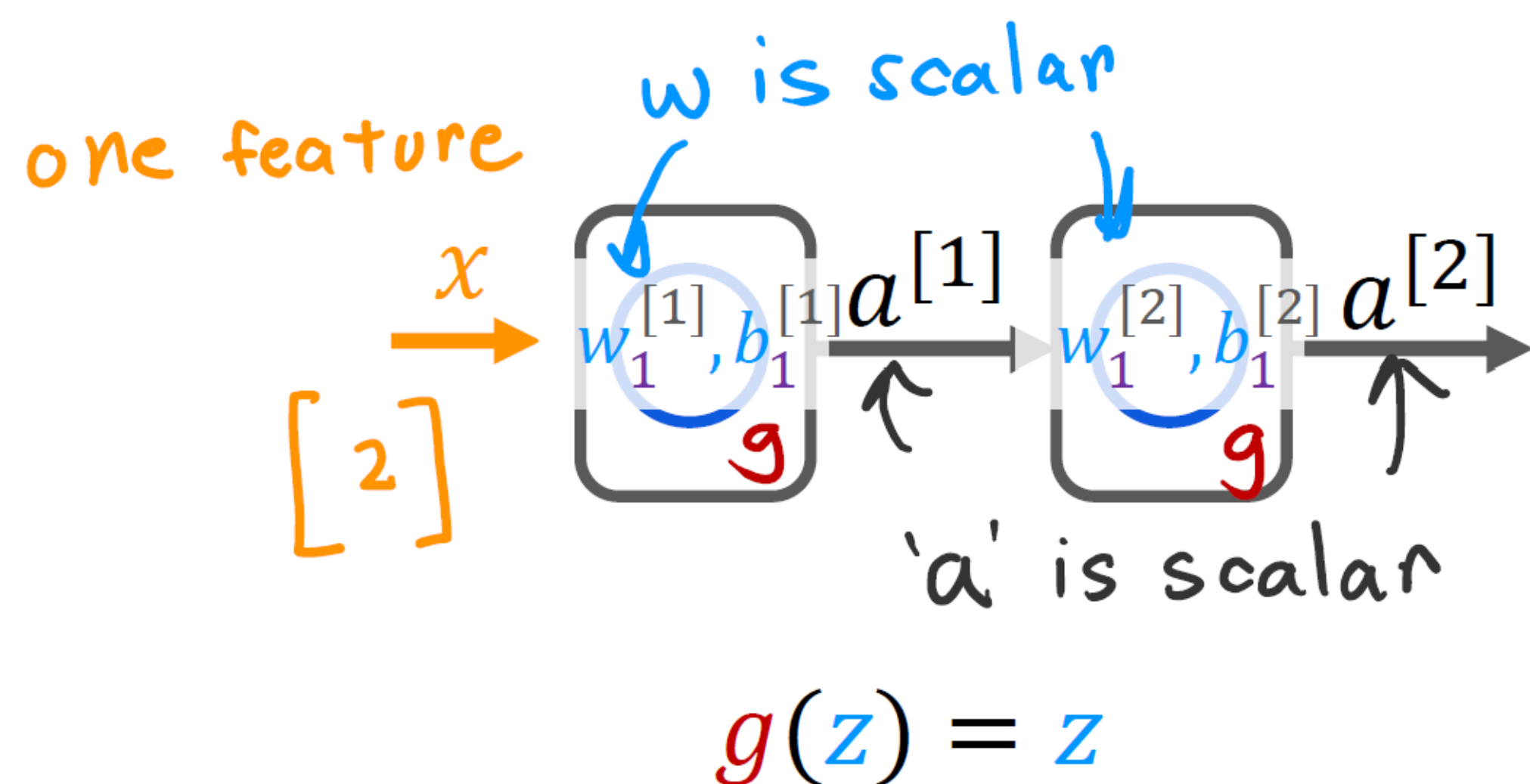
or 'linear'
or 'relu'

Why do we need activation functions



Why do we need activation functions

Linear Example



$$a^{[1]} = \underbrace{w_1^{[1]} x + b_1^{[1]}}$$

$$a^{[2]} = w_1^{[2]} a^{[1]} + b_1^{[2]}$$

$$= w_1^{[2]} (w_1^{[1]} x + b_1^{[1]}) + b_1^{[2]}$$

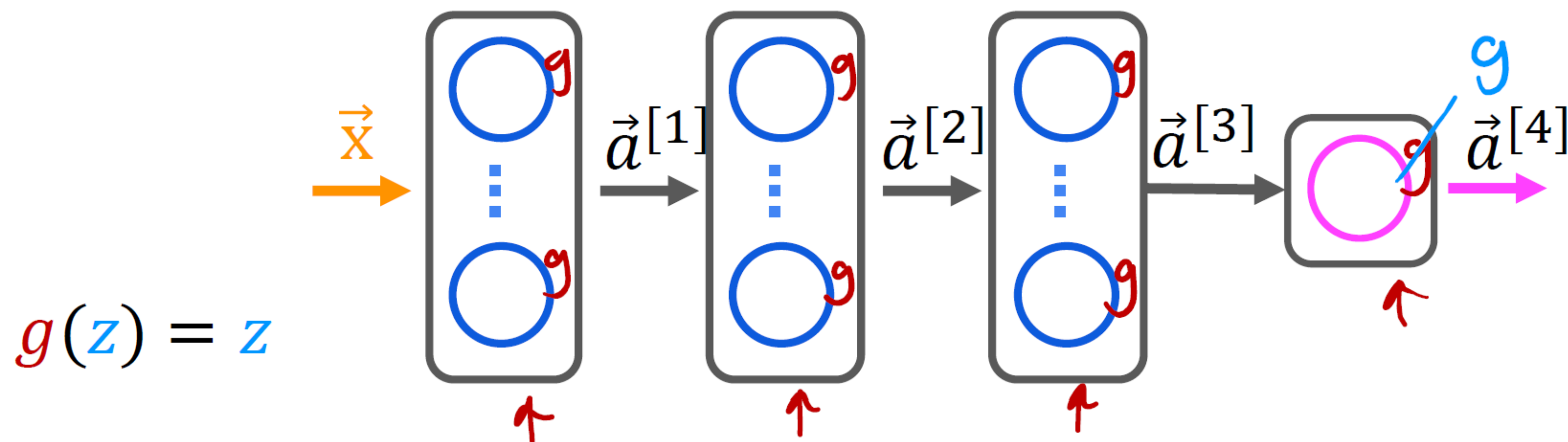
$$\vec{a}^{[2]} = \underbrace{\begin{pmatrix} \vec{w}_1^{[2]} & \vec{w}_1^{[1]} \end{pmatrix}}_w x + \underbrace{w_1^{[2]} b_1^{[1]} + b_1^{[2]}}_b$$

$$\vec{a}^{[2]} = w x + b$$

$$f(x) = wx + b \quad \text{linear regression}$$

Why do we need activation functions

Example



$$g(z) = z$$

$$\vec{a}^{[4]} = \vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]}$$

$$\vec{a}^{[4]} = \frac{1}{1 + e^{-(\vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]})}}$$

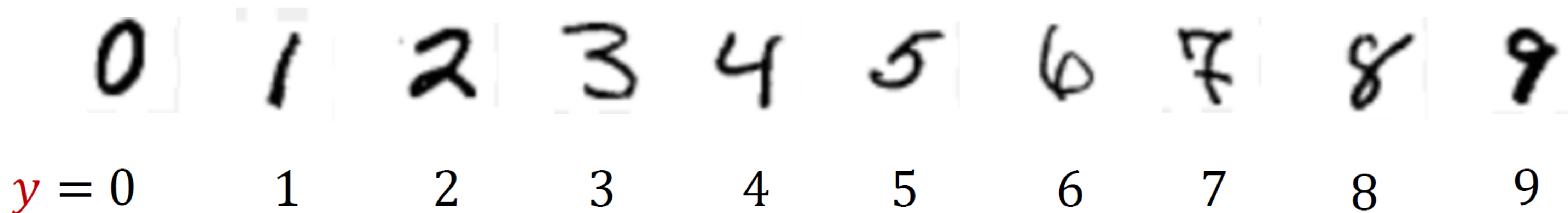
all linear (including output)
 ↳ equivalent to linear regression

output activation is sigmoid
 (hidden layers still linear)
 ↳ equivalent to logistic regression

Don't use linear activations in hidden layers (use ReLU)

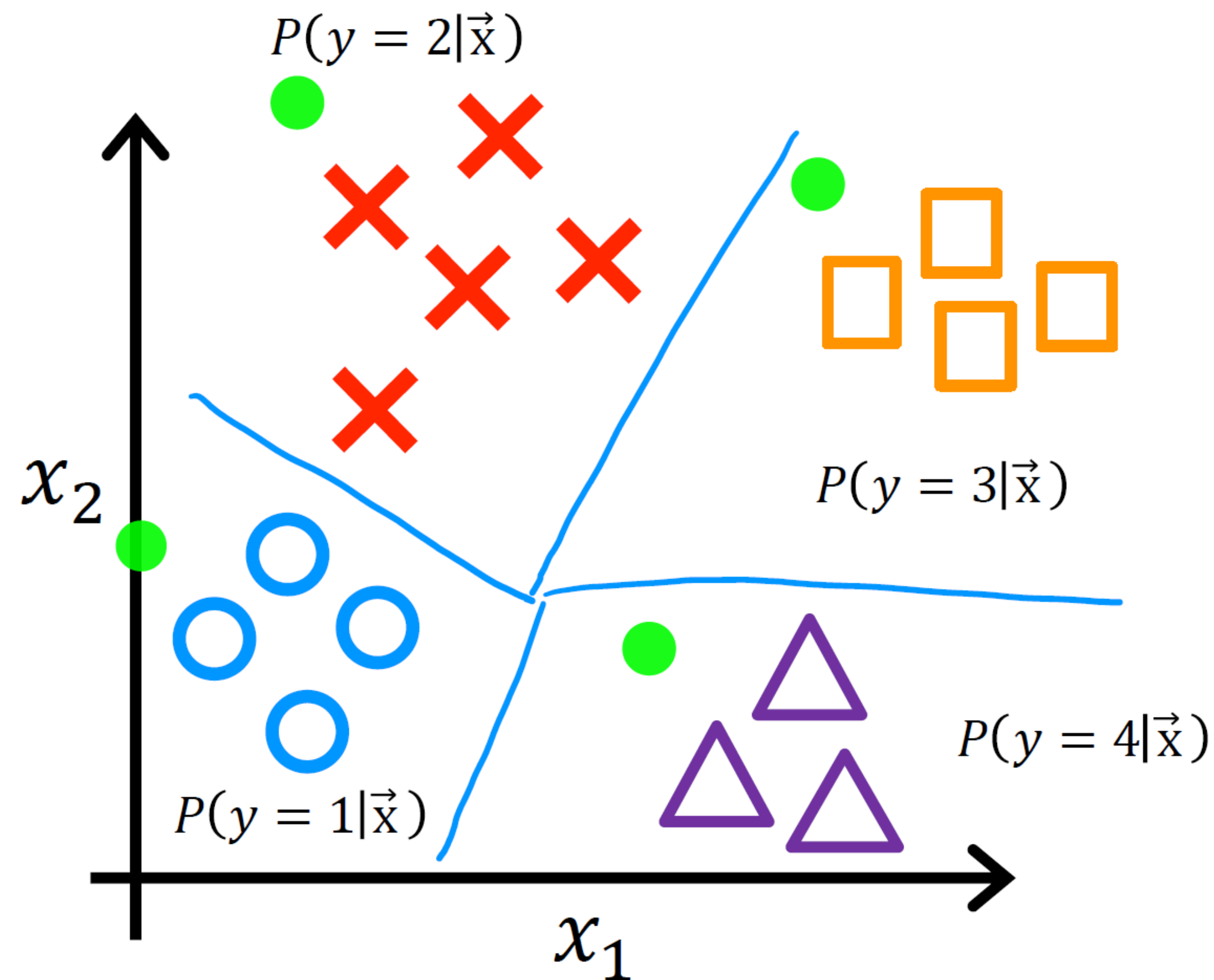
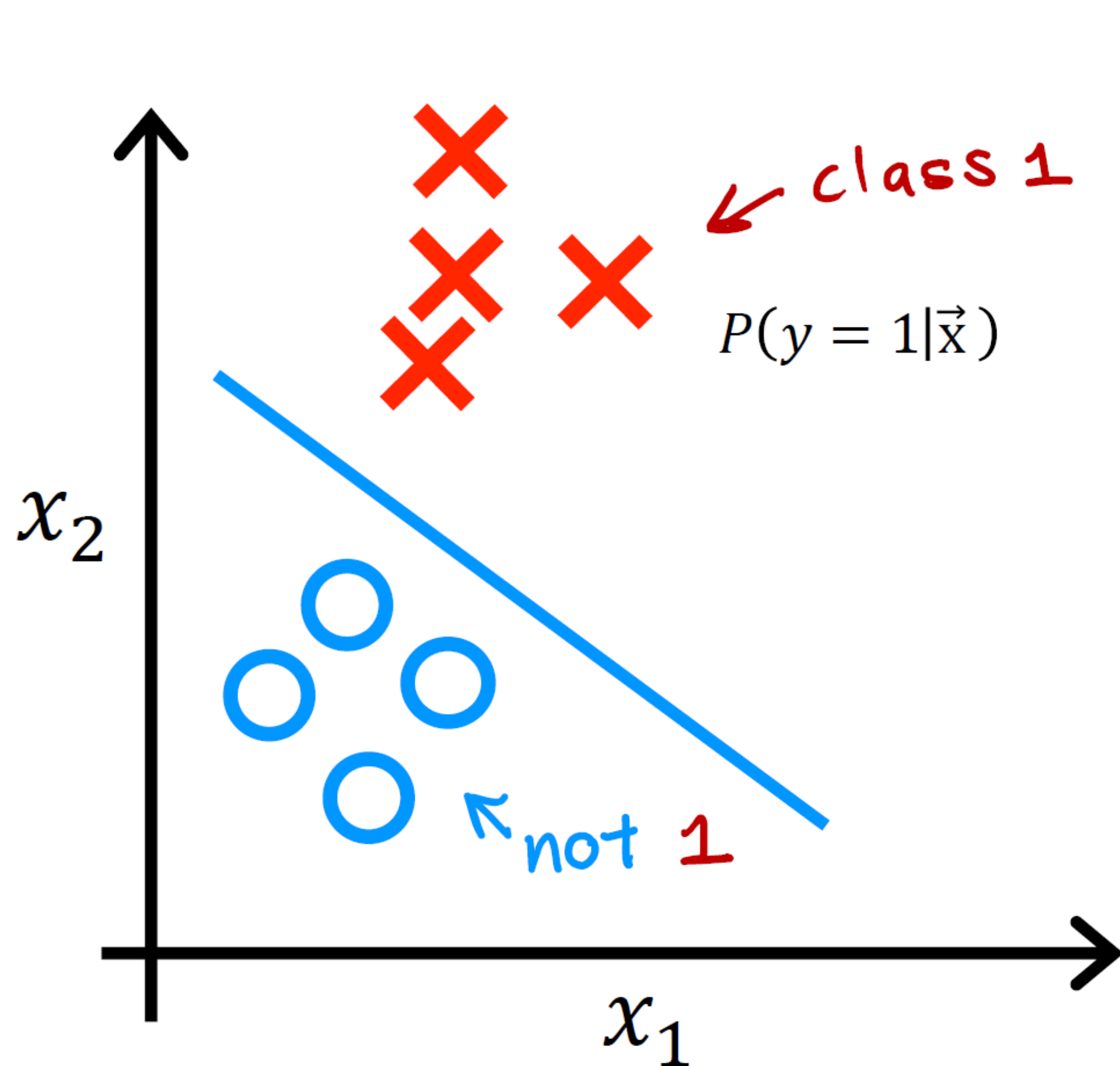
Multiclass

MNIST example



multiclass classification problem:
target y can take on more than two possible values

Multiclass classification example



Softmax

Logistic regression
(2 possible output values)

$$z = \vec{w} \cdot \vec{x} + b$$

$$\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.71$$

$$\circ a_2 = 1 - a_1 = P(y=0|\vec{x}) \quad 0.29$$

Softmax regression
(N possible outputs) $y=1,2,3,\dots,N$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$$

parameters $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

note: $a_1 + a_2 + \dots + a_N = 1$

Softmax regression (4 possible outputs) $y=1,2,3,4$

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=1|\vec{x}) \quad 0.30$$

$\times$ $\circ$ $\square$ $\triangle$

$$\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=2|\vec{x}) \quad 0.20$$

$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=3|\vec{x}) \quad 0.15$$

$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=4|\vec{x}) \quad 0.35$$

Cost

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 | \vec{x})$$

$$a_2 = 1 - a_1 = P(y = 0 | \vec{x})$$

$$\text{loss} = \underbrace{-y \log a_1}_{\text{if } y=1} - \underbrace{(1-y) \log(1-a_1)}_{\text{if } y=0}$$

$$J(\vec{w}, b) = \text{average loss}$$

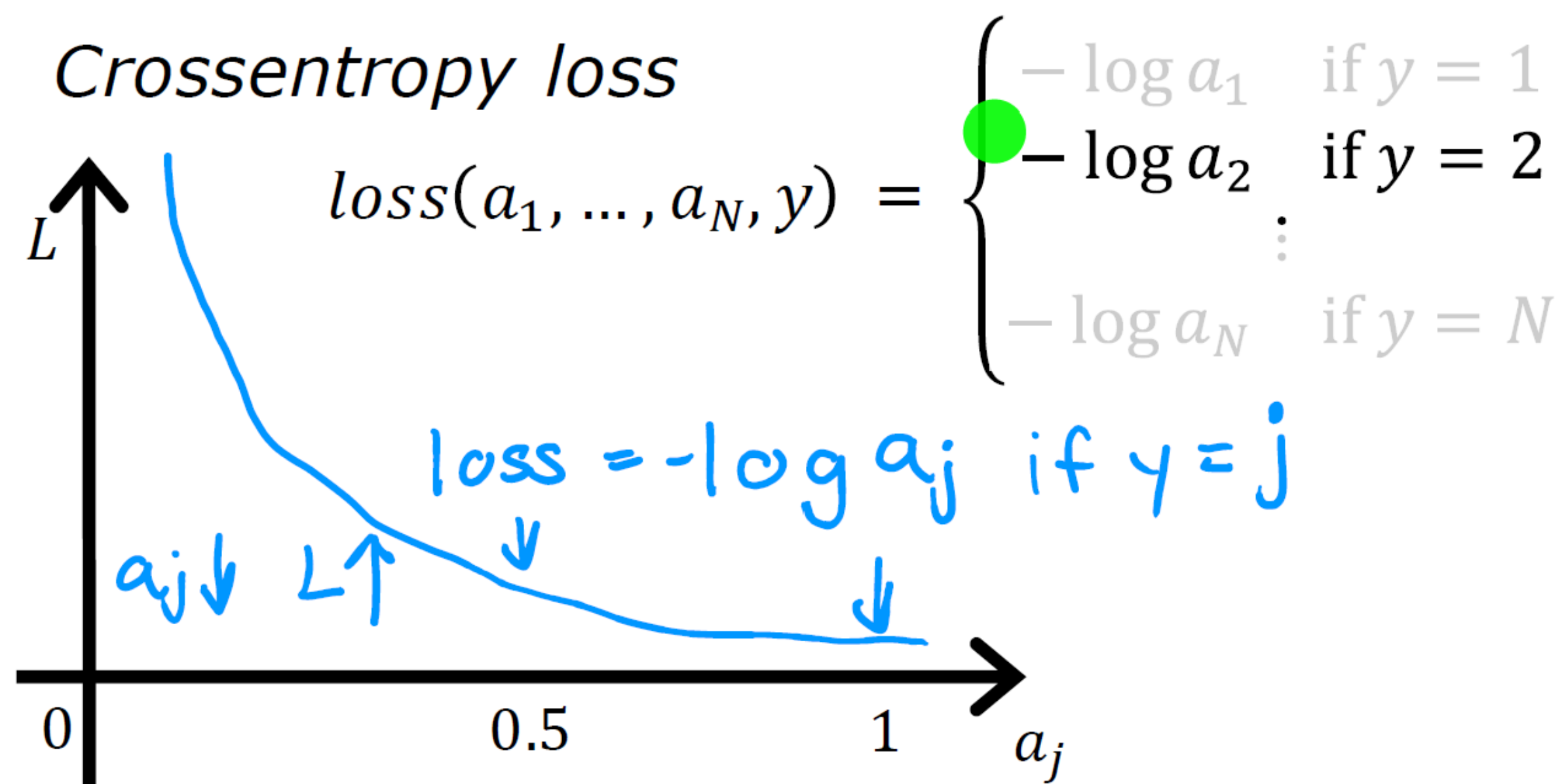
Softmax regression

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = 1 | \vec{x})$$

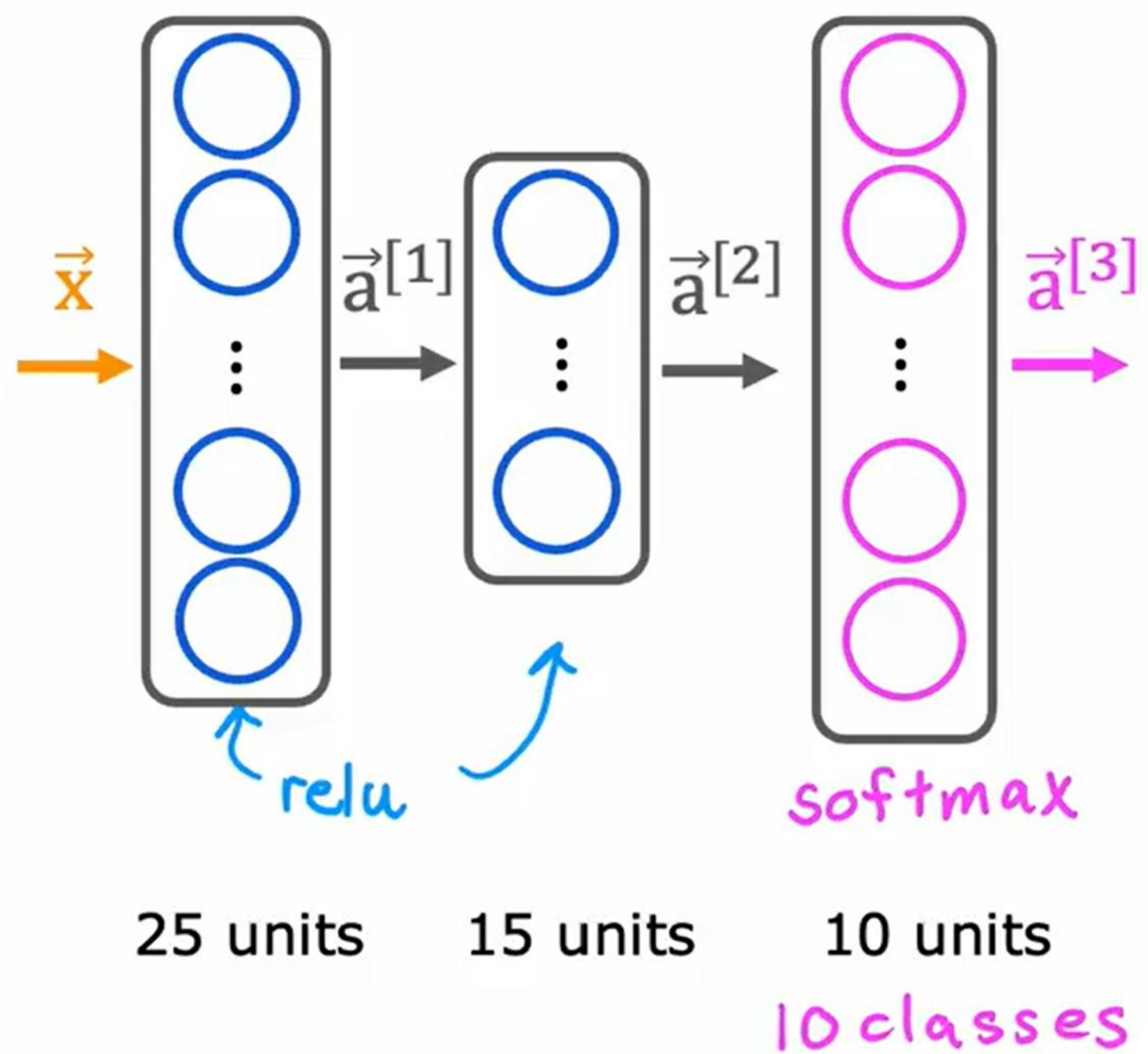
$$\vdots$$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = N | \vec{x})$$

Crossentropy loss



Neural Network with Softmax output



$$z_1^{[3]} = \vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]}$$

$$a_1^{[3]} = \frac{e^{z_1^{[3]}}}{e^{z_1^{[3]}} + \dots + e^{z_{10}^{[3]}}}$$

$$= P(y = 1 | \vec{x})$$

$$z_{10}^{[3]} = \vec{w}_{10}^{[3]} \cdot \vec{a}^{[2]} + b_{10}^{[3]}$$

$$a_{10}^{[3]} = \frac{e^{z_{10}^{[3]}}}{e^{z_1^{[3]}} + \dots + e^{z_{10}^{[3]}}}$$

$$= P(y = 10 | \vec{x})$$

logistic regression

$$a_1^{[3]} = g(z_1^{[3]}) \quad a_2^{[3]} = g(z_2^{[3]})$$

softmax

$$\vec{a}^{[3]} = (a_1^{[3]}, \dots, a_{10}^{[3]}) = g(z_1^{[3]}, \dots, z_{10}^{[3]})$$

MNIST with softmax

① specify the model

$$f_{\vec{w},b}(\vec{x}) = ?$$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
```

```
    Dense(units=25, activation='relu')
```

```
    Dense(units=15, activation='relu')
```

```
    Dense(units=10, activation='softmax')
```

② specify loss and cost

$$L(f_{\vec{w},b}(\vec{x}), y)$$

```
from tensorflow.keras.losses import
    SparseCategoricalCrossentropy
```

```
model.compile(loss= SparseCategoricalCrossentropy() )
```

```
model.fit (X,Y,epochs=100)
```

③ Train on data to minimize $J(\vec{w}, b)$

Note: better (recommended) version later.

Don't use the version shown here!

Improved implementation of softmax

Numerical Roundoff Errors

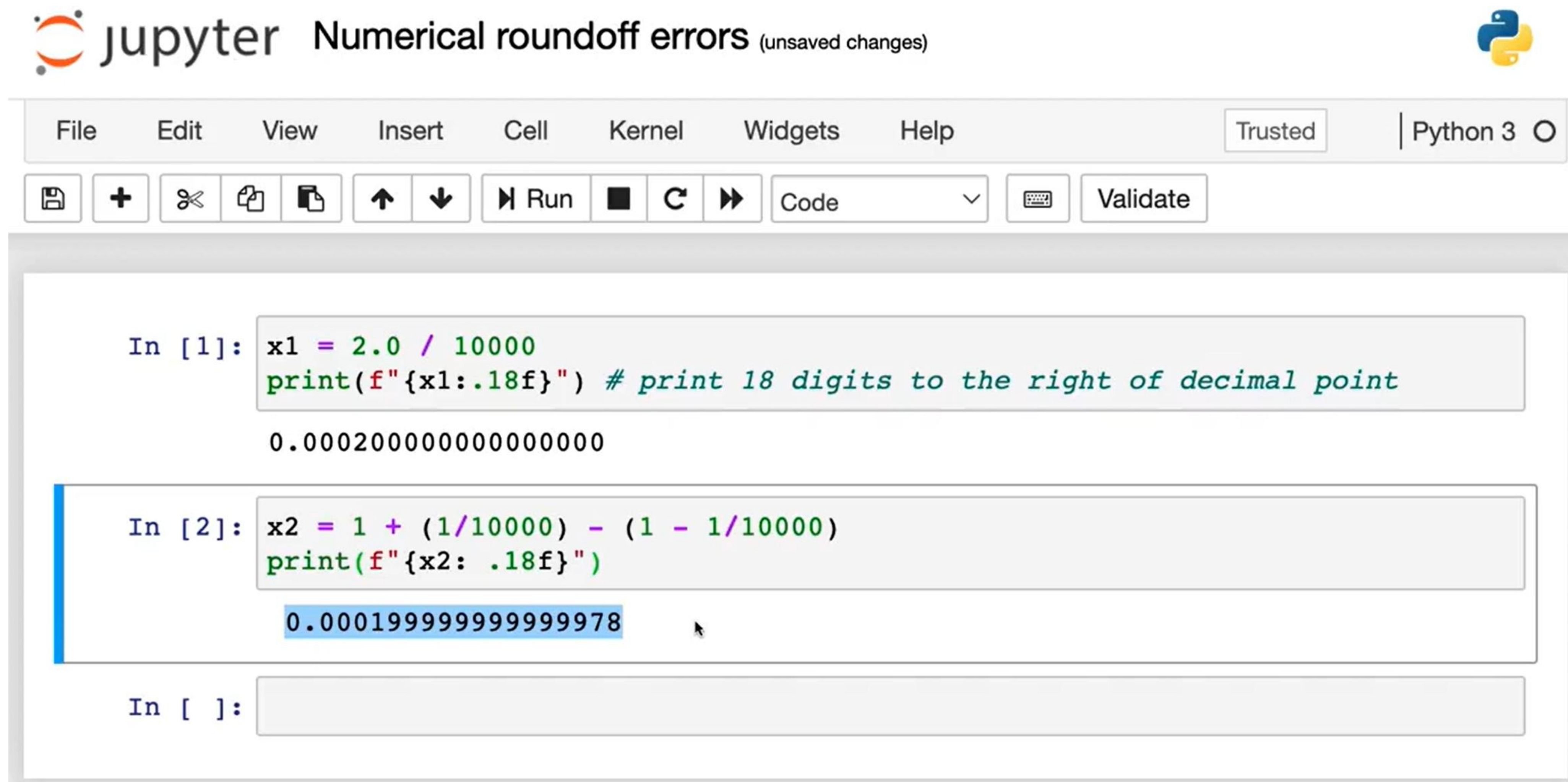
option 1

$$x = \frac{2}{10,000}$$

option 2

$$x = \underbrace{\left(1 + \frac{1}{10,000}\right)} - \underbrace{\left(1 - \frac{1}{10,000}\right)} =$$


Improved implementation of softmax



The image shows a Jupyter Notebook interface with the title "Numerical roundoff errors (unsaved changes)". The notebook contains two code cells. The first cell, labeled "In [1]:", calculates $x_1 = 2.0 / 10000$ and prints the result with 18 decimal places, showing `0.00020000000000000000`. The second cell, labeled "In [2]:", calculates $x_2 = 1 + (1/10000) - (1 - 1/10000)$ and prints the result with 18 decimal places, showing `0.0001999999999999978`. The output of the second cell is highlighted in blue. The interface includes a menu bar with options like File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. There is also a toolbar with icons for saving, adding cells, and running code. The Python version is indicated as Python 3.

jupyter Numerical roundoff errors (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

In [1]: `x1 = 2.0 / 10000`
`print(f"{x1:.18f}")` # print 18 digits to the right of decimal point

0.00020000000000000000

In [2]: `x2 = 1 + (1/10000) - (1 - 1/10000)`
`print(f"{x2: .18f}")`

0.0001999999999999978

In []:

Improved implementation of softmax

More numerically accurate implementation of logistic loss:

$$1 + \frac{1}{10,000} \quad 1 - \frac{1}{10,000}$$

Logistic regression:

$$\boxed{a} = g(z) = \frac{1}{1 + e^{-z}}$$

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'), 'linear'
    Dense(units=1, activation='sigmoid')
])
```

Original loss

$$loss = -y \log(\boxed{a}) - (1 - y) \log(1 - \boxed{a})$$

~~model.compile(loss=BinaryCrossEntropy())~~

model.compile(loss=BinaryCrossEntropy(from_logits=True))

More accurate loss (in code)

$$loss = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

Improved implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
    Dense(units=25, activation='relu')
    Dense(units=15, activation='relu')
    Dense(units=10, activation='softmax')
```

'linear'

~~model.compile(loss=SparseCategoricalCrossEntropy())~~

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

model.compile(loss=SparseCrossEntropy(from_logits=True))



Improved implementation of softmax

```

model    import tensorflow as tf
         from tensorflow.keras import Sequential
         from tensorflow.keras.layers import Dense
         model = Sequential([
             Dense(units=25, activation='relu')
             Dense(units=15, activation='relu')
             Dense(units=10, activation='linear') ])

loss     from tensorflow.keras.losses import
         SparseCategoricalCrossentropy

         model.compile(..., loss=SparseCategoricalCrossentropy(from_logits=True) )

fit      model.fit(X, Y, epochs=100)

predict  logits = model(X) ← not  $a_1 \dots a_{10}$  is  $z_1 \dots z_{10}$ 
         f_x = tf.nn.softmax(logits)

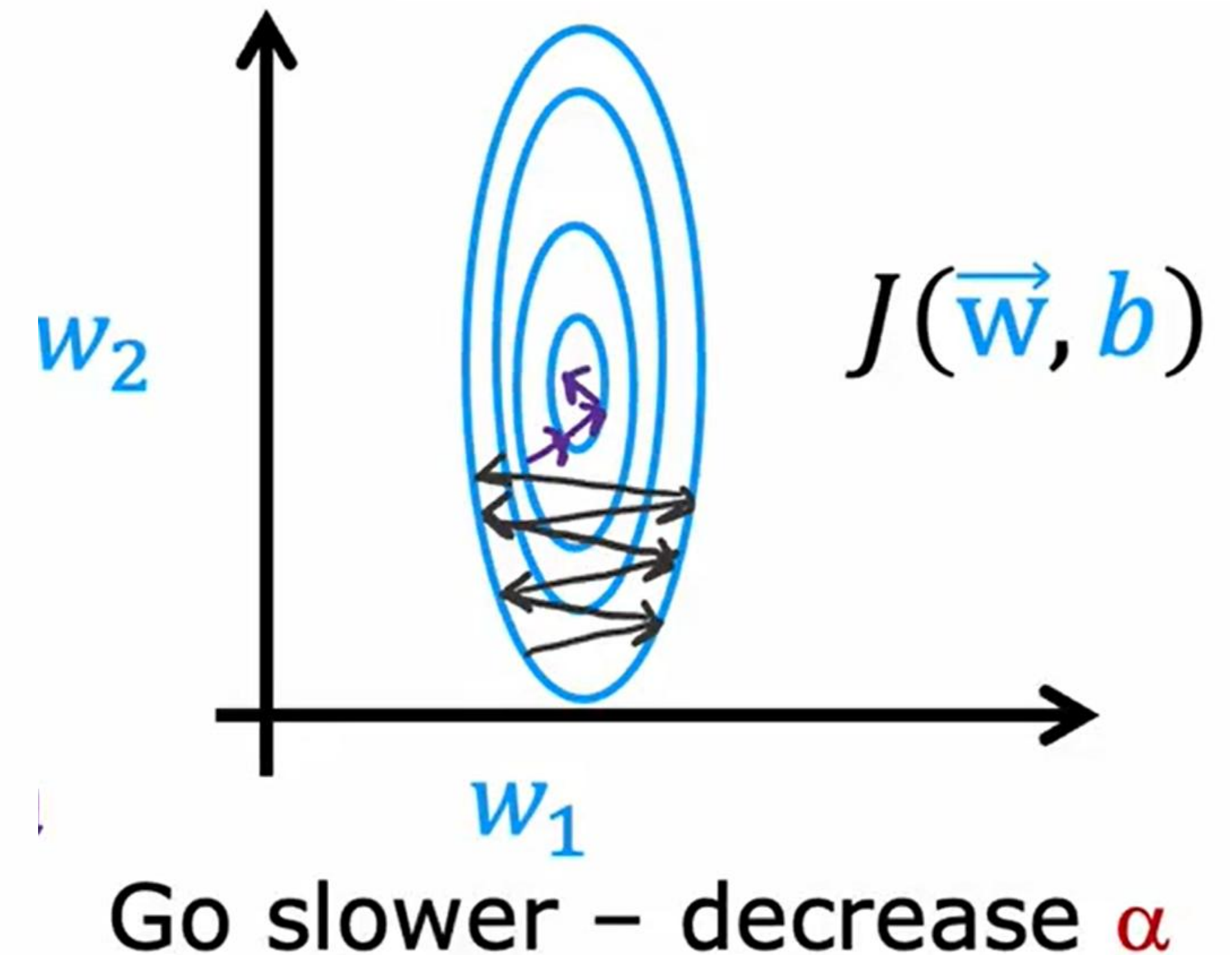
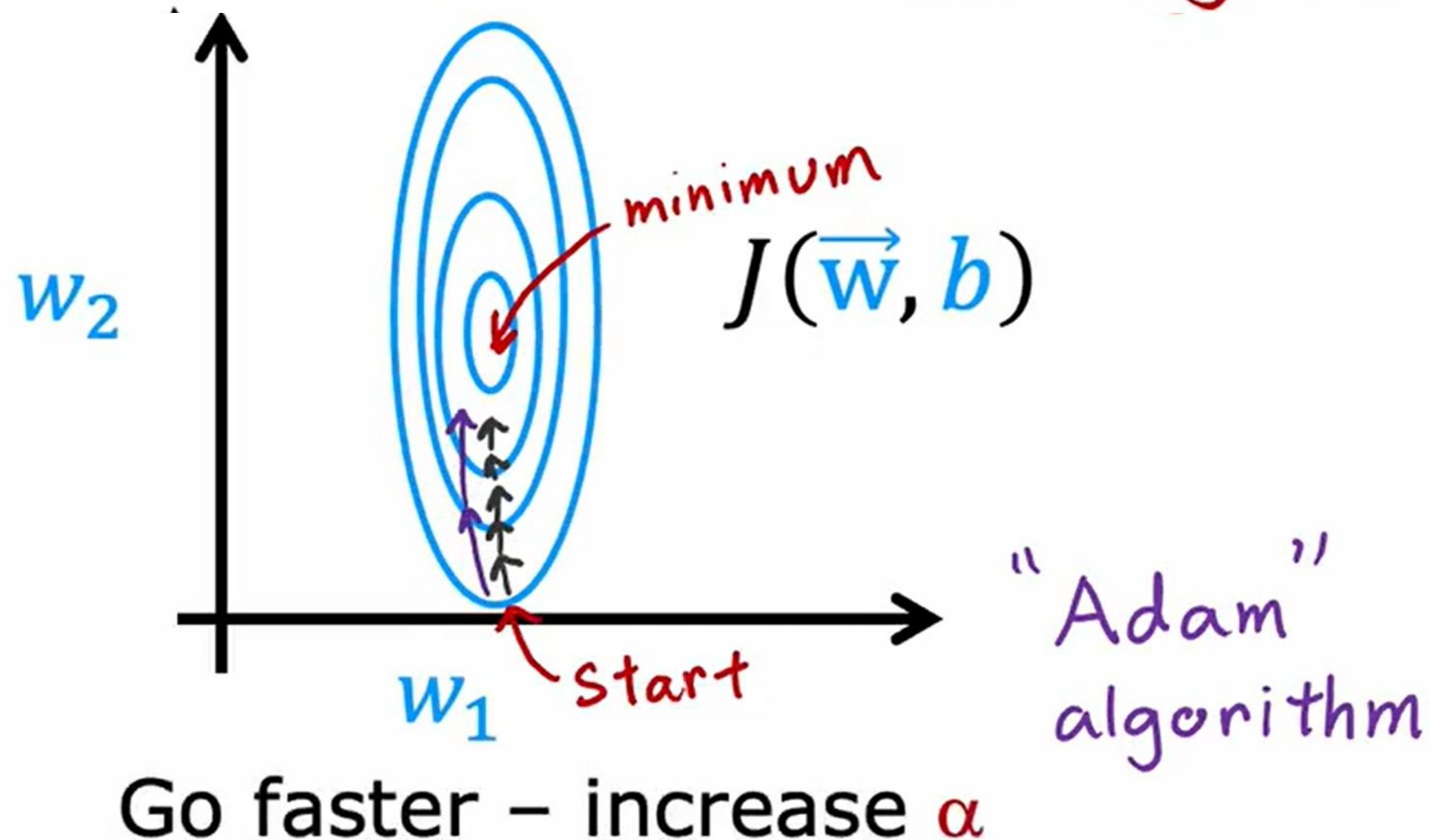
```


Advanced Optimization

Gradient Descent

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

learning rate

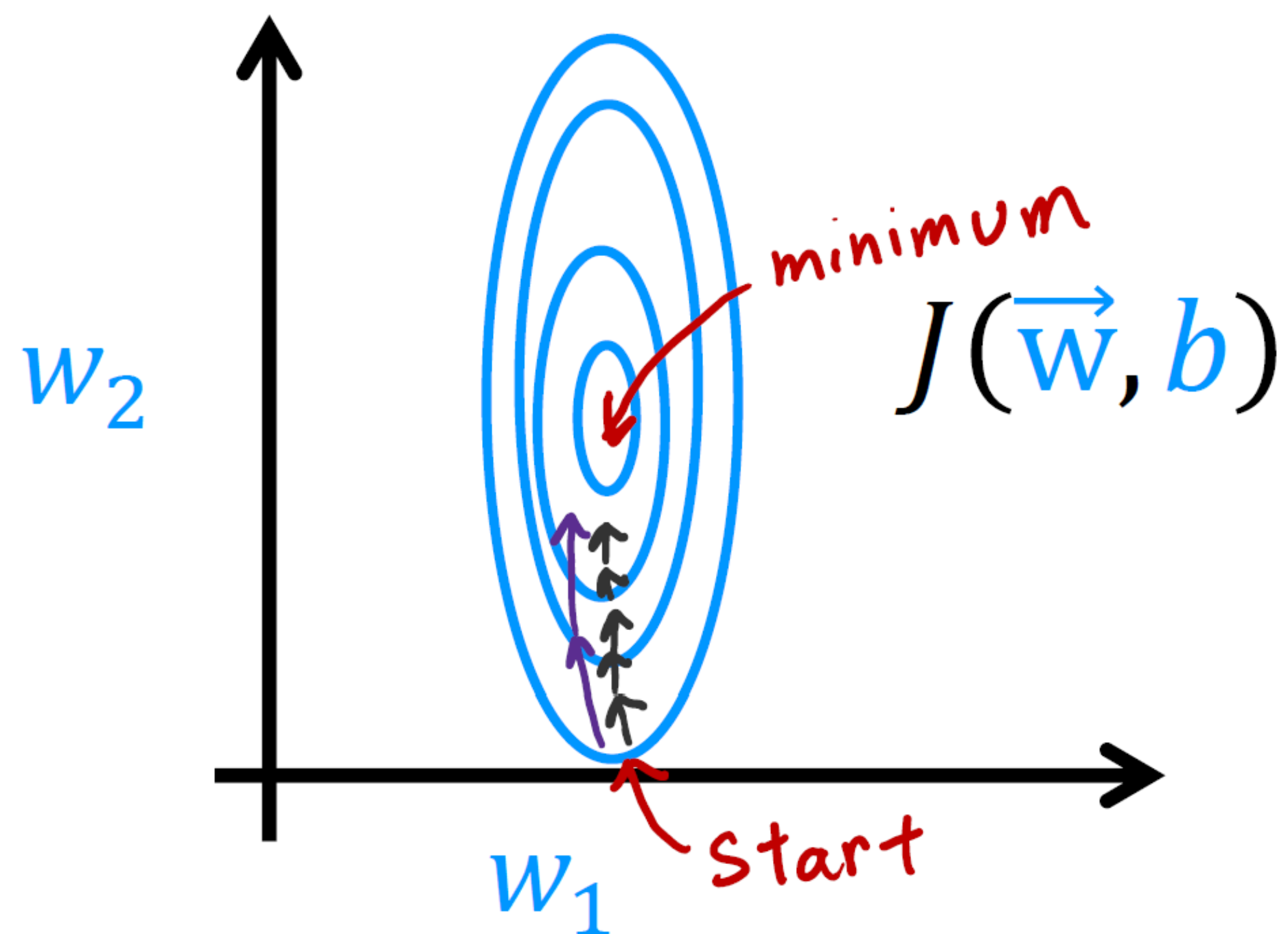


Adam Algorithm Intuition

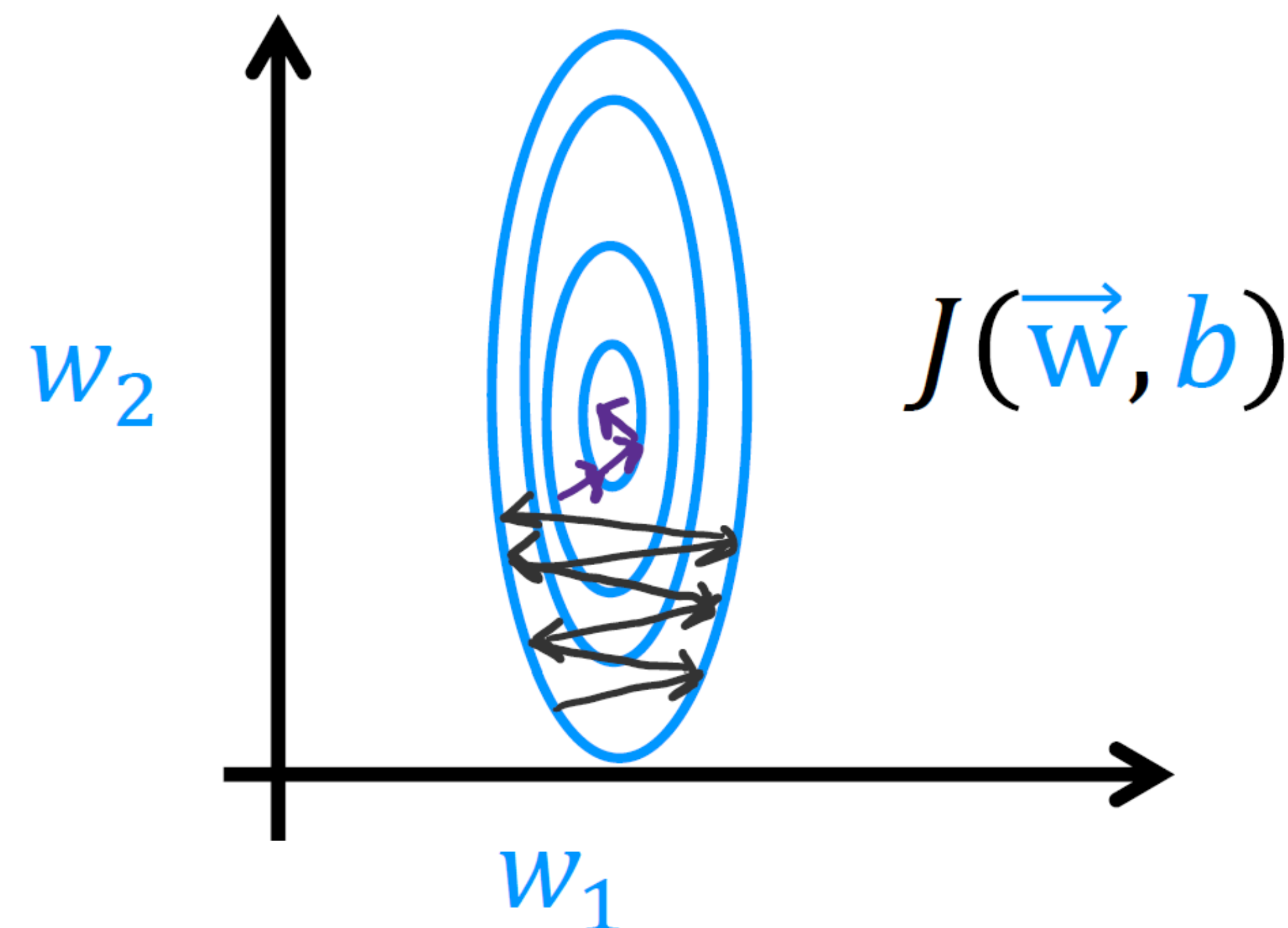
Adam: Adaptive Moment estimation *not just one α*

$$\begin{aligned}w_1 &= w_1 - \underbrace{\alpha_1}_{\text{adaptive}} \frac{\partial}{\partial w_1} J(\vec{w}, b) \\&\vdots \\w_{10} &= w_{10} - \underbrace{\alpha_{10}}_{\text{adaptive}} \frac{\partial}{\partial w_{10}} J(\vec{w}, b) \\b &= b - \underbrace{\alpha_{11}}_{\text{adaptive}} \frac{\partial}{\partial b} J(\vec{w}, b)\end{aligned}$$

Adam Algorithm Intuition



If w_j (or b) keeps moving in same direction, increase α_j .



If w_j (or b) keeps oscillating, reduce α_j .

MNIST Adam

model

```
model = Sequential([  
    tf.keras.layers.Dense(units=25, activation='sigmoid')  
    tf.keras.layers.Dense(units=15, activation='sigmoid')  
    tf.keras.layers.Dense(units=10, activation='linear')  
])
```

compile

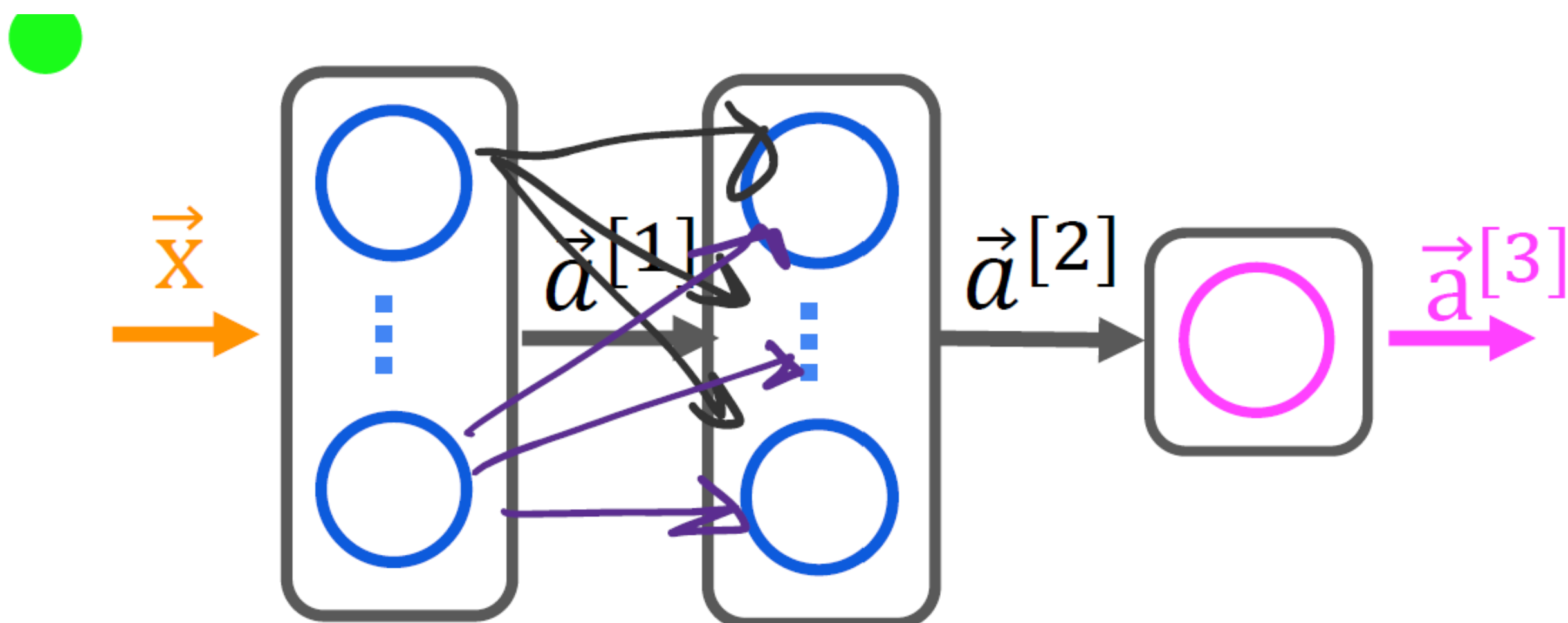
```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

$$\alpha = 10^{-3} = 0.001$$

fit

```
model.fit(X, Y, epochs=100)
```

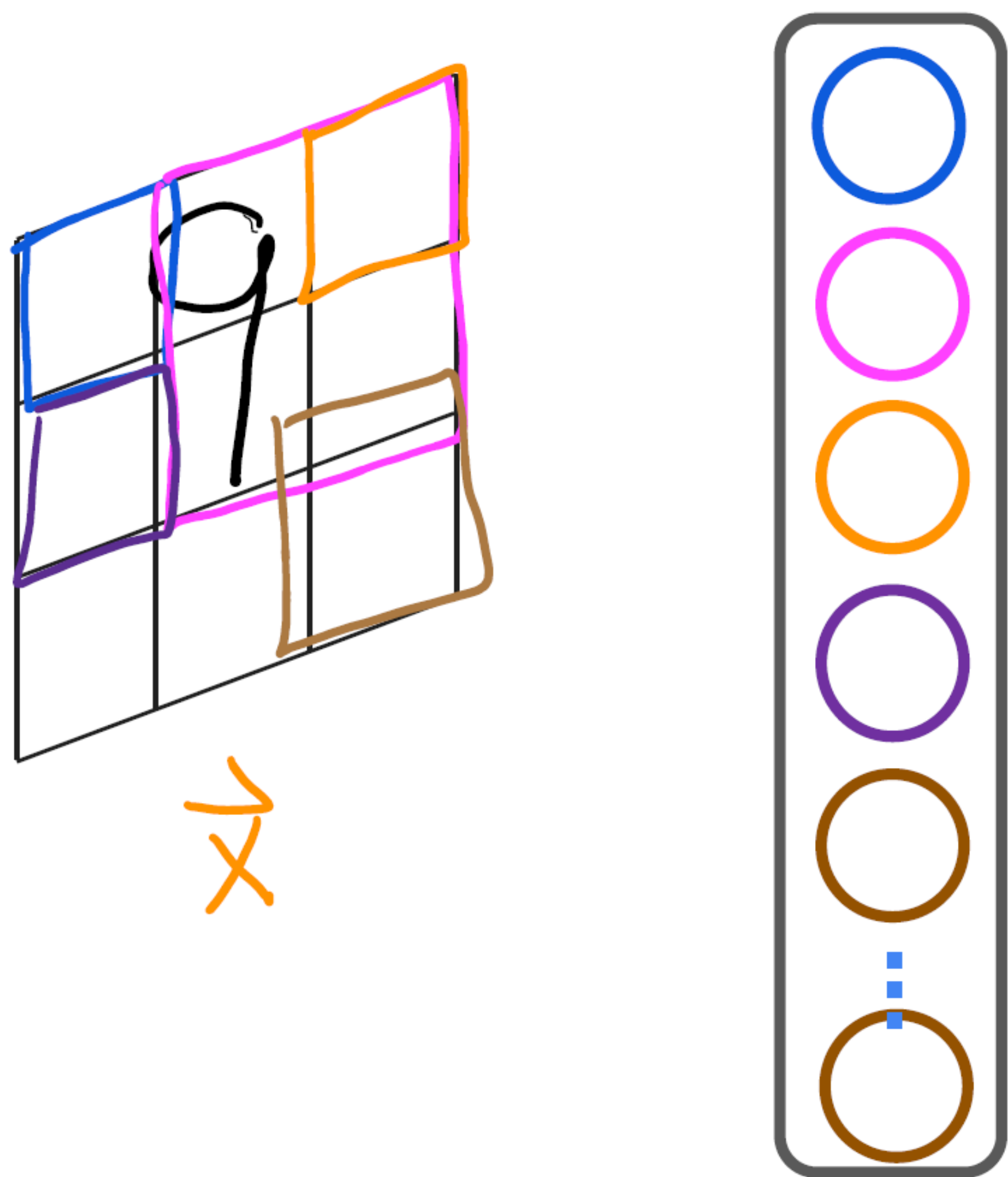
Dense Layer



Each neuron output is a function of all the activation outputs of the previous layer.

- $$\vec{a}_1^{[2]} = g \left(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]} \right)$$

Convolutional Layer

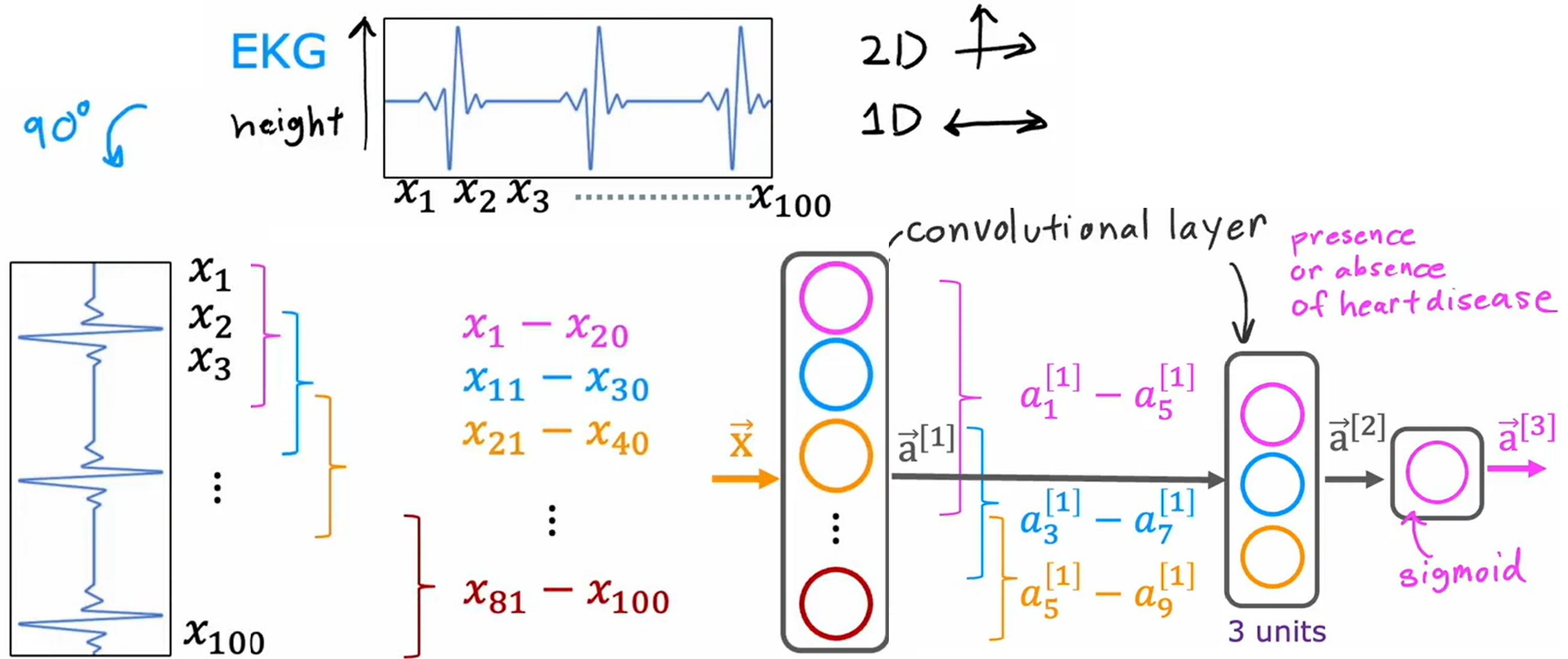


Each Neuron only looks at part of the previous layer's inputs.

Why?

- Faster computation
- Need less training data (less prone to overfitting)

Convolutional Neural Network



Thank You

Any questions?

kushol@iut-dhaka.edu

Source & Special Thanks to Andrew Ng (Coursera)
Machine Learning Course 2

<https://www.youtube.com/@Deeplearningai>

<https://www.youtube.com/watch?v=ggWLvh484hs&list=PLyoNSC4BT4eVpykPF0Yx8C1Zs50XtD17L>

<https://medium.com/@sushmita2310/12-types-of-activation-functions-in-neural-networks-a-comprehensive-guide-a441ecef439>